

C3 AI Coding Prep — Worked Problems (Full Hit-List)

Contents

C3 AI LeetCode — Real Solution Walkthroughs	2
1. Fraction to Recurring Decimal (LeetCode 166) — Medium	2
2. K-diff Pairs in an Array (LeetCode 532) — Medium	8
3. Koko Eating Bananas (LeetCode 875) — Medium	11
4. Unique Paths (LeetCode 62) — Medium	13
5. Coin Change (LeetCode 322) — Medium	14
6. Valid Parentheses (LeetCode 20) — Easy	16
7. Trapping Rain Water (LeetCode 42) — Hard	17
8. Rotting Oranges (LeetCode 994) — Medium	19
9. Merge Intervals (LeetCode 56) — Medium	20
10. LRU Cache (LeetCode 146) — Medium	22
11. Subarray Sum Equals K (LeetCode 560) — Medium	23
Shared imports (problems 12+)	24
12. Find the Index of the First Occurrence in a String (LeetCode 28) — Easy	25
13. Find Peak Element (LeetCode 162) — Medium	26
14. Largest Number (LeetCode 179) — Medium	28
15. Identify the Largest Outlier in an Array (LeetCode 3371) — Medium	29
16. Custom Sort String (LeetCode 791) — Medium	30
17. Design HashMap (LeetCode 706) — Easy	32
18. Rotate Image (LeetCode 48) — Medium	33
19. Design Tic-Tac-Toe (LeetCode 348) — Medium	35
20. Longest Arithmetic Subsequence of Given Difference (LeetCode 1218) — Medium	36
21. Merge Sorted Array (LeetCode 88) — Easy	37
22. Maximum Sum Circular Subarray (LeetCode 918) — Medium	39
23. Accounts Merge (LeetCode 721) — Medium	40
24. Combination Sum II (LeetCode 40) — Medium	42
25. Reverse Linked List (LeetCode 206) — Easy	44
26. First Unique Character in a String (LeetCode 387) — Easy	45
27. Two City Scheduling (LeetCode 1029) — Medium	46
28. Count Unhappy Friends (LeetCode 1583) — Medium	47
29. Find K Closest Elements (LeetCode 658) — Medium	49
30. Range Sum of BST (LeetCode 938) — Easy	50
31. Search in Rotated Sorted Array (LeetCode 33) — Medium	51
32. Word Search (LeetCode 79) — Medium	53
33. Remove All Occurrences of a Substring (LeetCode 1910) — Medium	54
34. Longest Increasing Subsequence (LeetCode 300) — Medium	56
35. Move Pieces to Obtain a String (LeetCode 2337) — Medium	57
36. Search a 2D Matrix (LeetCode 74) — Medium	58
37. Search Suggestions System (LeetCode 1268) — Medium	60

38. Binary Search Tree Iterator (LeetCode 173) — Medium	61
39. Minimum Operations to Make Character Frequencies Equal (LeetCode 3389) — Hard	63
40. Random Pick with Weight (LeetCode 528) — Medium	64
41. Design Hit Counter (LeetCode 362) — Medium	66
42. Remove Stones to Minimize the Total (LeetCode 1962) — Medium	67
43. Remove Nth Node From End of List (LeetCode 19) — Medium	69
44. Simplify Path (LeetCode 71) — Medium	70
45. Zero Array Transformation I (LeetCode 3355) — Medium	71
46. Sqrt(x) (LeetCode 69) — Easy	72
47. All Nodes Distance K in Binary Tree (LeetCode 863) — Medium	74
48. The kth Factor of n (LeetCode 1492) — Medium	75
49. Continuous Subarray Sum (LeetCode 523) — Medium	77
50. Best Time to Buy and Sell Stock IV (LeetCode 188) — Hard	78
51. Palindromic Substrings (LeetCode 647) — Medium	79
52. Generate Parentheses (LeetCode 22) — Medium	81
53. Container With Most Water (LeetCode 11) — Medium	82
54. Spiral Matrix (LeetCode 54) — Medium	83
55. Gas Station (LeetCode 134) — Medium	85
56. Zigzag Iterator (LeetCode 281) — Medium	86
57. Group the People Given the Group Size They Belong To (LeetCode 1282) — Medium	88
58. Top K Frequent Words (LeetCode 692) — Medium	89
Study note	90
How to study (so this sticks)	90

C3 AI LeetCode — Real Solution Walkthroughs

How to read this: For each problem we name a **method**, then **use that method on a concrete example**, showing the data structures after every step. Only after you see the method working do we write code.

This file covers the full C3 AI hit-list (~46+ problems): deep walkthroughs for the highest-frequency ones first, then the remaining problems in the same method → example → code shape.

If your editor still shows an old short file, close the tab and reopen `text.md` from disk.

1. Fraction to Recurring Decimal (LeetCode 166) — Medium

Method we will use

Long division + a hash map from remainder → index in the answer

- Long division produces one decimal digit at a time.
- The **remainder** (leftover) after each step decides the next digit.
- If the same remainder appears again, the digit sequence will repeat from the place we first saw that remainder.
- So the map tells us *where to put the opening parenthesis*.

We are not “using a hash map in the abstract.” We are using it as a **memory of remainders we already processed during long division**.

Problem in one line

Given `numerator` and `denominator`, return the decimal as a string. If the fractional part repeats, wrap the repeating block in `()`.

Examples: - $1/2 \rightarrow "0.5"$ (stops) - $1/3 \rightarrow "0.(3)"$ (pure repeat) - $1/6 \rightarrow "0.1(6)"$ (delayed repeat) - $-50/8 \rightarrow "-6.25"$

Why floats / “just divide” fail (so we need this method)

- `str(1/6)` in Python might look like `0.16666666666666666`.
- That does **not** tell you the repeating block starts after the 1.
- So we must **simulate** the division ourselves and detect the cycle with the remainder map.

Using the method on 1 / 6 (full walkthrough)

We will keep two things updated at every step:

- `res` = list of pieces of the answer string
- `seen` = dict { remainder : index in res where that remainder's digit starts }
- `rem` = current remainder

Setup

What we do	Why (method)	State after
<code>numerator=1, denominator=6</code> both positive $\rightarrow$ no minus	sign via XOR later	<code>res=[]</code>
<code>n=1, d=6</code>	work with absolutes	
Integer part: $1 // 6 = 0 \rightarrow$ append "0"	long division before the decimal	<code>res=["0"]</code>
<code>rem = 1 % 6 = 1</code>	leftover after integer part	<code>rem=1</code>
<code>rem 0</code> $\rightarrow$ append "."	there is a fractional part	<code>res=["0", "."]</code>
<code>seen = {}</code>	start remainder memory	<code>seen={}</code>

Fractional loop — apply long division + map Iteration A — rem is 1

1. Is 1 in `seen`? **No**.
2. Method says: remember this remainder's digit position $\rightarrow$ `seen[1] = len(res) = 2`.
3. Long division: `rem *= 10` $\rightarrow$ 10.

4. Digit = $10 // 6 = 1 \rightarrow$ append "1".

5. New rem = $10 \% 6 = 4$.

State now:

```
res = ["0", ".", "1"]
      index: 0    1    2
seen = {1: 2}      # remainder 1 produced the digit at index 2
rem = 4
```

Iteration B — rem is 4

1. Is 4 in seen? **No**.

2. Save seen[4] = len(res) = 3.

3. rem *= 10 $\rightarrow$ 40.

4. Digit = $40 // 6 = 6 \rightarrow$ append "6".

5. New rem = $40 \% 6 = 4$.

State now:

```
res = ["0", ".", "1", "6"]
      index: 0    1    2    3
seen = {1: 2, 4: 3}
rem = 4
```

Iteration C — rem is 4 again $\leftarrow$ cycle detected by the method

1. Is 4 in seen? **Yes** — at index 3.

2. Method says: the repeating block starts at index 3.

3. Insert "(" at index 3, append ")".

4. Stop.

State now:

```
res = ["0", ".", "1", "(", "6", ")"]
join  $\rightarrow$  "0.1(6)"
```

That is the whole solution: **we used long division to make digits, and we used the remainder map to know where the cycle starts.**

Same method on 1 / 3 (pure repeat)

Step	rem	seen?	digit / action	res after
start	1	—	write 0.	0.
1	1	no	save seen [1]=2; digit 3 ; rem stays 1	0.3
2	1	yes @2	insert (at index 2, then)	0.(3)

After step	seen map
start	{}
1	{1: 2}
2	(cycle found — stop)

Answer: "0.(3)".

Same method on 1 / 2 (terminates — rem becomes 0)

Step	rem	action	res
start	1	write 0.	0.
1	1 → 10/2 digit 5, rem 0	write 5	0.5
stop	0	rem==0 ends loop, no parentheses	"0.5"

The method still works: cycle detection never fires because rem hits 0.

Same method on -50 / 8 (sign + longer fraction)

1. Signs differ → start with "-".
2. 50 // 8 = 6 → "-6", rem 2.
3. Write "." → "-6.".
4. rem 2 → 20/8 digit 2, rem 4 → "-6.2".
5. rem 4 → 40/8 digit 5, rem 0 → "-6.25".
6. rem 0 → stop. Answer "-6.25".

Only now: code that mirrors the walkthrough above

```
def fractionToDecimal(numerator: int, denominator: int) -> str:
    # --- edge ---
    if numerator == 0:
        return "0"

    res = []

    # --- sign (same as walkthrough setup) ---
    if (numerator < 0) ^ (denominator < 0):
        res.append("-")

    n, d = abs(numerator), abs(denominator)

    # --- integer part ---
    res.append(str(n // d))
    rem = n % d
    if rem == 0:
        return "".join(res)    # like 4/2 -> "2"

    # --- fractional part: THIS is where we use the method ---
    res.append(".")
    seen = {}    # rem -> index in res (exactly as in the tables above)

    while rem:
        # METHOD STEP: have we seen this remainder before?
        if rem in seen:
            # METHOD STEP: cycle starts at the saved index
            res.insert(seen[rem], "(")
            res.append(")")
            break

        # METHOD STEP: remember where this rem's digit will sit
        seen[rem] = len(res)

        # METHOD STEP: one long-division tick
        rem *= 10
        res.append(str(rem // d))
        rem %= d

    return "".join(res)
```

Mapping: walkthrough line → code line

Walkthrough action on 1/6	Code
Write integer 0	<code>res.append(str(n // d))</code>
rem = 1, write .	<code>rem = n % d</code> then <code>res.append(".")</code>
First time rem=1 → save index 2	<code>seen[rem] = len(res)</code>
10/6 write 1, rem=4	<code>rem*=10; append</code> <code>rem//d; rem%=d</code>
First time rem=4 → save index 3	same
40/6 write 6, rem=4	same
rem=4 already in seen → insert (at 3	<code>res.insert(seen[rem],</code> <code>"("); append ")"</code>

Complexity

Time	$O(\text{length of answer})$. At most d different remainders before a repeat, so $O(d)$.
Space	$O(d)$ for <code>seen</code> + answer.

Alternate — complexity trick

Trick: Math shrink + fractional buffer (no `list.insert`)

- Cancel `gcd(n, d)` first so the remainder map is over a smaller modulus.
- After canceling, strip all factors of 2 and 5 from `d`. If nothing remains (`d == 1`), the decimal **terminates** — you never need a remainder map (space for cycle detection drops to $O(1)$).
- Build fractional digits in a separate `frac` list and **slice** when the cycle starts. Avoid `res.insert(...)` (shifts the whole list, $O(p)$ per insert → $O(p^2)$ if abused).

```
from math import gcd

def fractionToDecimal(numerator, denominator):
    if numerator == 0:
        return "0"
    sign = "-" if (numerator < 0) ^ (denominator < 0) else ""
    n, d = abs(numerator), abs(denominator)
    g = gcd(n, d)
    n, d = n // g, d // g

    integer, rem = divmod(n, d)
    if rem == 0:
        return sign + str(integer)
```

```

# Math: if d's only prime factors are 2 and/or 5, decimal terminates
dd = d
while dd % 2 == 0:
    dd //= 2
while dd % 5 == 0:
    dd //= 5
terminates = dd == 1

frac, first = [], {}
while rem:
    if not terminates and rem in first:
        i = first[rem]
        return sign + str(integer) + "." + "".join(frac[:i]) + "(" + "".join(frac[i:]) + ")"
    if not terminates:
        first[rem] = len(frac)
    rem *= 10
    frac.append(str(rem // d))
    rem %= d
return sign + str(integer) + "." + "".join(frac)

```

Time	$O(p)$ fractional digits (same order as main; no $O(p)$ middle shifts).
Space	$O(1)$ extra map when it terminates; else $O(\text{period})$ $O(d)$ where d is d with factors 2 and 5 removed.
vs main	Math tells you when the map is unnecessary; slicing beats <code>insert</code> for cycle parentheses.

What to say in the interview

“I’ll simulate long division. I store each remainder and the index of the digit it produced. When a remainder repeats, I insert parentheses at the stored index — that’s exactly where the repeating block starts.”

Similar / follow-ups

- Similar: Divide Two Integers (29)
- Variation: return `True` if the decimal terminates (after canceling factors 2 and 5 from the denominator, nothing remains)

2. K-diff Pairs in an Array (LeetCode 532) — Medium

Method we will use

Frequency map (Counter), then case-split on k

- We care about **values**, not indices.
- Build `count[value] = how many times it appears`.
- If $k > 0$: a unique pair exists for value x iff $x + k$ also appears (check each x once).

- If $k == 0$: a unique pair exists for x iff $\text{count}[x] \geq 2$.

Using the method on `nums = [3,1,4,1,5]`, $k = 2$

```
nums: 3, 1, 4, 1, 5
```

```
count = {
  3: 1,
  1: 2,
  4: 1,
  5: 1
}
```

Step 1 — apply the frequency map

Step 2 — apply the $k > 0$ rule with the map Walk each **unique key** once; ask “is $x+k$ a key?”

x	$x+k = x+2$	in count?	pair?
3	5	yes	(3,5)
1	3	yes	(1,3)
4	6	no	
5	7	no	

Answer = **2**.

Notice we did **not** also check $x-k$. If we did, (1,3) would be counted twice (once from 1 and once from 3). The method “only look upward ($x+k$)” prevents that.

Step 3 — same method when $k == 0$ Example: `nums=[1,1,1,2]`, $k=0$

```
count = {1: 3, 2: 1}
```

Rule for $k=0$: count keys with freq $\geq 2 \rightarrow$ only 1 $\rightarrow$ answer **1**.

(If you wrongly used the $k > 0$ rule, $1+0$ is always in the map and you’d double-count nonsense.)

Code that mirrors those steps

```
from collections import Counter

def findPairs(nums, k):
    # METHOD STEP 1: frequency map
    count = Counter(nums)

    if k < 0:
```

```

    return 0 # absolute difference can't be negative

# METHOD STEP 2a: k == 0 case
if k == 0:
    # pair = same value appearing at least twice
    return sum(1 for freq in count.values() if freq > 1)

# METHOD STEP 2b: k > 0 case - only look at x+k
ans = 0
for x in count:
    if (x + k) in count:
        ans += 1
return ans

```

Walkthrough code

Method action	Code
Build frequency map	<code>count = Counter(nums)</code>
<code>k=0</code> : need duplicates	<code>freq > 1</code>
<code>k>0</code> : check partner <code>x+k</code>	<code>(x + k) in count</code>

Alternate — complexity trick

Trick: Set fast path

- For `k > 0`, multiplicity is irrelevant: iterate a set once and test `x + k`; only `k == 0` needs frequencies.
- Use it when the interviewer asks whether a full `Counter` is necessary for every branch.

```

from collections import Counter

def findPairs(nums, k):
    if k < 0:
        return 0
    if k == 0:
        return sum(v > 1 for v in Counter(nums).values())
    values = set(nums)
    return sum(x + k in values for x in values)

```

Time	$O(n)$ expected.
Space	$O(u)$, for u distinct values.
vs main	Avoids <code>Counter</code> values and counting overhead on the <code>k > 0</code> path.

3. Koko Eating Bananas (LeetCode 875) — Medium

Method we will use

Binary search on the answer (the eating speed)

- Define `ok(speed) = True` if Koko finishes all piles in `h` hours at that speed.
- `ok` is False for small speeds, then becomes True and stays True.
- Binary search finds the **smallest** speed where `ok` is True.
- Search range is `[1, max(piles)]`, not indices in the array.

Using the method on `piles=[3,6,7,11]`, `h=8`

Helper the method needs: hours for a given speed Hours for speed `s = ceil(3/s)+ceil(6/s)+ceil(7/s)+ceil(11/s)`

speed	hours	ok? (8?)
1	3+6+7+11=27	no
3	1+2+3+4=10	no
4	1+2+2+3=8	yes
6	1+1+2+2=6	yes
11	1+1+1+1=4	yes

Feasibility pattern: no, no, no, YES, YES, YES...

Method: binary search the first YES → answer 4.

Apply binary search step by step Start `lo=1`, `hi=11` (max pile).

lo	hi	mid	hours(mid)	ok?	update	meaning
1	11	6	6	yes	hi=6	mid works; try slower
1	6	3	10	no	lo=4	need faster
4	6	5	8	yes	hi=5	try slower
4	5	4	8	yes	hi=4	try slower
4	4	—	—	—	stop	answer 4

We used the method: each `mid` is a candidate speed; `ok(mid)` decides which half of the speed range dies.

Code that mirrors the walkthrough

```
def minEatingSpeed(piles, h):
    def hours_needed(speed):
        # METHOD helper: total hours at this speed
        total = 0
        for p in piles:
            total += (p + speed - 1) // speed # ceil(p/speed)
        return total

    # METHOD: binary search the speed range
    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        if hours_needed(mid) <= h: # ok(mid)?
            hi = mid # try slower
        else:
            lo = mid + 1 # need faster
    return lo
```

Alternate — complexity trick

Trick: Early feasibility cutoff

- While testing a speed, stop summing as soon as required hours exceed h ; ceiling division is $(p + \text{speed} - 1) // \text{speed}$.
- Use it when failed binary-search probes contain many piles and can be rejected early.

```
def minEatingSpeed(piles, h):
    def works(speed):
        hours = 0
        for pile in piles:
            hours += (pile + speed - 1) // speed
            if hours > h:
                return False
        return True

    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        if works(mid): hi = mid
        else: lo = mid + 1
    return lo
```

Time	$O(n \log M)$, $M = \text{max pile}$; rejected probes may stop early.
Space	$O(1)$.
vs main	Keeps the optimal search bound while pruning unnecessary feasibility work.

4. Unique Paths (LeetCode 62) — Medium

Method we will use

DP / memo on grid cells: $\text{ways}(r,c) = \text{ways}(r+1,c) + \text{ways}(r,c+1)$

- Every path is only right/down moves.
- The number of ways from a cell is a fixed subproblem.
- Compute it once (memo) or bottom-up (DP row).

Using the method on $m=3, n=2$

Grid cells labeled with “ways from here to end”:

```

Start (0,0) ----> (0,1)
  |               |
  v               v
(1,0) ----> (1,1)
  |               |
  v               v
(2,0) ----> (2,1) End

```

Apply the recurrence from the end backward

- $\text{ways}(2,1) = 1$ (already at end)
- $\text{ways}(2,0) = \text{ways}(2,1) + \text{ways}(3,0) = 1 + 0 = 1$ (only right)
- $\text{ways}(1,1) = \text{ways}(2,1) + \text{ways}(1,2) = 1 + 0 = 1$ (only down)
- $\text{ways}(1,0) = \text{ways}(2,0) + \text{ways}(1,1) = 1 + 1 = 2$
- $\text{ways}(0,1) = \text{ways}(1,1) + \text{ways}(0,2) = 1 + 0 = 1$
- $\text{ways}(0,0) = \text{ways}(1,0) + \text{ways}(0,1) = 2 + 1 = 3$

Answer **3**. The method filled each cell using only cells below/right.

Same method as a rolling 1D row Start top row all 1s (only move right along the top): $[1, 1]$

Process next row: for j from $1..n-1$: $\text{row}[j] += \text{row}[j-1]$ - After row 2: $[1, 2]$ - After row 3: $[1, 3] \rightarrow$ answer 3

Code

```

def uniquePaths(m, n):
    # METHOD as 1D DP: row[j] = ways to current cell in this row
    row = [1] * n
    for _ in range(m - 1):
        for j in range(1, n):
            # from left + from above (above is old row[j] before update)
            row[j] += row[j - 1]
    return row[-1]

```

C3 note: unmemoized recursion is about $O(2^{(m+n)})$, **not** $O(2^{(m*n)})$. Return values from dfs so @lru_cache works — no global counter.

Alternate — complexity trick

Trick: Binomial path count

- Every path is an ordering of $m-1$ down moves and $n-1$ right moves, so compute one binomial coefficient.
- Use it when there are no blocked cells; obstacles destroy the direct combination formula.

```
def uniquePaths(m, n):
    total = m + n - 2
    choose = min(m - 1, n - 1)
    ans = 1
    for i in range(1, choose + 1):
        ans = ans * (total - choose + i) // i
    return ans
```

Time $O(\min(m, n))$.

Space $O(1)$.

vs main Beats $O(mn)$ DP time and its row/table storage.

5. Coin Change (LeetCode 322) — Medium

Method we will use

1D DP: $dp[x]$ = fewest coins to make amount x

- $dp[0] = 0$
 - For each amount a , try each coin $c \leq a$:
 $dp[a] = \min(dp[a], dp[a-c] + 1)$
 - That means: “use coin c as the last coin, on top of the best way to make $a-c$.”
-

Using the method on $coins=[1,2,5]$, $amount=11$

Initialize: $dp[0]=0$, everything else inf .

Fill amounts 1..11 (showing the updates that matter):

a	try coins	best $dp[a]$	meaning
1	$1 \rightarrow dp[0]+1$	1	one 1
2	$1 \rightarrow 2; 2 \rightarrow 1$	1	one 2
3	$1 \rightarrow 2; 2 \rightarrow 2$	2	e.g. 1+2

a	try coins	best dp[a]	meaning
4	...	2	2+2
5	5→1	1	one 5
6	5→2, 2→2, 1→3	2	5+1
10	5→2	2	5+5
11	5→ dp[6]+1=3; 2→...; 1→...	3	5+5+1

Answer **3**. We used the method: each amount's answer was built only from smaller amounts.

Code

```
def coinChange(coins, amount):
    dp = [0] + [float("inf")] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                # METHOD: last coin is c
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != float("inf") else -1
```

Alternate — complexity trick

Trick: Layered amount BFS

- Treat each reachable amount as a node and each added coin as one edge; the first layer reaching `amount` uses the fewest coins.
- Use it when the target may be reached in very few coins, allowing an early stop; DP is usually simpler otherwise.

```
from collections import deque

def coinChange(coins, amount):
    q, seen, steps = deque([0]), {0}, 0
    while q:
        for _ in range(len(q)):
            cur = q.popleft()
            if cur == amount:
                return steps
            for coin in coins:
                nxt = cur + coin
                if nxt <= amount and nxt not in seen:
                    seen.add(nxt)
                    q.append(nxt)
        steps += 1
    return -1
```

Time	$O(\text{amount} \cdot \text{len}(\text{coins}))$ worst case.
Space	$O(\text{amount})$.
vs main	Can stop before filling the whole DP range when the optimum has few coins.

6. Valid Parentheses (LeetCode 20) — Easy

Method we will use

Stack of unmatched opening brackets

- Scan left $\rightarrow$ right.
- Opener $\rightarrow$ push.
- Closer $\rightarrow$ must match the **top** of the stack (most recent unmatched opener).
- End with empty stack.

Using the method on "{[]}"

char	action (method)	stack after
{	opener $\rightarrow$ push	[{]
[	opener $\rightarrow$ push	[{, [
]	closer $\rightarrow$ pop must be [	[{]
}	closer $\rightarrow$ pop must be {	[] empty

Empty $\rightarrow$ **true**.

Using the method on "([)]" (fails)

char	action	stack
(	push	(
[	push	([
)	pop expects [but map says need ($\rightarrow$ mismatch $\rightarrow$ false	

The method failed at the third character because the top wasn't the matching opener.

Code

```
def isValid(s):
    stack = []
    pairs = {"(": ")", "[": "]", "{": "}"
    for ch in s:
        if ch in pairs: # closer
            if not stack or stack.pop() != pairs[ch]:
                return False
        else:
            stack.append(ch)
    return not stack
```

Alternate — complexity trick

Trick: Single-type balance

- With only (and), a counter replaces the stack; reject immediately if a prefix balance becomes negative.
- Use only when there is one bracket type; matching multiple types still requires a stack.

```
def isValidSingleType(s):
    balance = 0
    for ch in s:
        balance += 1 if ch == "(" else -1
        if balance < 0:
            return False
    return balance == 0
```

Time	$O(n)$.
Space	$O(1)$.
vs main	Removes the $O(n)$ stack under the single-bracket constraint.

7. Trapping Rain Water (LeetCode 42) — Hard

Method we will use

Two pointers + running max on each side

- Water at an index is limited by the **shorter** of the tallest bars on its left and right.
- Point l at start, r at end.
- Whichever side has the smaller height is the bottleneck for that index — settle its water, move that pointer.

Using the method on $[0,1,0,2,1,0,1,3,2,1,2,1]$ (abbreviated)

Idea of one step: - If $height[l] < height[r]$, left is shorter. - Update $maxL = \max(maxL, height[l])$.
 - Water added at l is $maxL - height[l]$. - Then $l += 1$.

Symmetric when right is shorter.

You never need a full leftmax/rightmax array if you always process the smaller side first — that **is** the method.

Code

```
def trap(height):
    l, r = 0, len(height) - 1
    maxL = maxR = ans = 0
    while l < r:
        if height[l] < height[r]:
            maxL = max(maxL, height[l])
            ans += maxL - height[l]    # METHOD: left is bottleneck
            l += 1
        else:
            maxR = max(maxR, height[r])
            ans += maxR - height[r]
            r -= 1
    return ans
```

Alternate — complexity trick

Trick: Prefix maxima

- Precompute the highest wall on each side, making each cell's trapped water a direct $\min(\text{left}, \text{right}) - \text{height}$ calculation.
- Use it when clarity and easy verification matter more than the two-pointer solution's space advantage.

```
def trap(height):
    n = len(height)
    if n < 3: return 0
    left, right = height[:], height[:]
    for i in range(1, n):
        left[i] = max(left[i - 1], height[i])
    for i in range(n - 2, -1, -1):
        right[i] = max(right[i + 1], height[i])
    return sum(min(left[i], right[i]) - height[i] for i in range(n))
```

Time	$O(n)$.
Space	$O(n)$.
vs main	Same linear time but a clearer invariant; intentionally trades away $O(1)$ space.

8. Rotting Oranges (LeetCode 994) — Medium

Method we will use

Multi-source BFS

- Every rotten orange is a source at minute 0.
- Each BFS layer = one minute of simultaneous infection.
- Track remaining fresh count.

Using the method on a small grid

minute 0:		minute 1:		minute 2:
2 1 1		2 2 1		2 2 2
1 1 0	→	2 1 0	→	2 2 0
0 1 1		0 1 1		0 2 1

Method application: 1. Queue all cells with 2 at the start (here only top-left). 2. Layer 1: infect its fresh neighbors → they become 2 and enter the queue. 3. Layer 2: those infect the next ring. 4. Continue until queue empty or no fresh left. 5. Minutes = number of layers processed while fresh were infected.

DFS from one orange is the **wrong method** because infection is parallel.

Code

```
from collections import deque

def orangesRotting(grid):
    R, C = len(grid), len(grid[0])
    q = deque()
    fresh = 0
    for r in range(R):
        for c in range(C):
            if grid[r][c] == 2:
                q.append((r, c))  # METHOD: all sources at t=0
            elif grid[r][c] == 1:
                fresh += 1

    minutes = 0
    while q and fresh > 0:
        for _ in range(len(q)):  # METHOD: one minute layer
            r, c = q.popleft()
            for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
                nr, nc = r+dr, c+dc
                if 0 <= nr < R and 0 <= nc < C and grid[nr][nc] == 1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    q.append((nr, nc))
        minutes += 1
```

```
return minutes if fresh == 0 else -1
```

Alternate — complexity trick

Trick: Grid timestamps

- Write minute values directly into fresh-orange cells, so the grid itself is both visited state and distance.
- Use it when mutating the input is allowed; otherwise keep a separate visited structure.

```
from collections import deque

def orangesRotting(grid):
    q = deque((r, c) for r in range(len(grid))
              for c in range(len(grid[0])) if grid[r][c] == 2)
    last = 2
    while q:
        r, c = q.popleft()
        last = max(last, grid[r][c])
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]) and grid[nr][nc] == 1:
                grid[nr][nc] = grid[r][c] + 1
                q.append((nr, nc))
    return -1 if any(1 in row for row in grid) else last - 2
```

Time	$O(\text{rows} \cdot \text{cols})$.
Space	$O(\text{rows} \cdot \text{cols})$ queue worst case; $O(1)$ visited storage.
vs main	Eliminates a separate visited set by encoding BFS time in place.

9. Merge Intervals (LeetCode 56) — Medium

Method we will use

Sort by start, then merge into the last interval in the output

Using the method on $[[1,3],[2,6],[8,10],[15,18]]$

Step 1 — sort by start (already sorted).

Step 2 — sweep:

next interval	last in out	overlaps? (next.start last.end)	action	out after
start with [1,3]	—	—	seed	[[1,3]]

next interval	last in out	overlaps? (next.start last.end)	action	out after
[2,6]	[1,3]	2 3 yes	extend end to max(3,6)=6	[[1,6]]
[8,10]	[1,6]	8 6 no	append	[[1,6], [8,10]]
[15,18]	[8,10]	no	append	[[1,6], [8,10], [15,18]]

That is the entire method applied.

Code

```
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    out = [intervals[0][:]]
    for s, e in intervals[1:]:
        if s <= out[-1][1]:          # METHOD: overlaps last
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

Alternate — complexity trick

Trick: In-place sort and coalesce

- Sort the input itself and append only disjoint intervals; overwrite the last output end when intervals overlap.
- Use it when input mutation is permitted and output storage is not counted as auxiliary space.

```
def merge(intervals):
    intervals.sort()
    out = []
    for start, end in intervals:
        if not out or start > out[-1][1]:
            out.append([start, end])
        else:
            out[-1][1] = max(out[-1][1], end)
    return out
```

Time	$O(n \log n)$.
Space	$O(1)$ auxiliary beyond sorting internals and output.
vs main	Avoids a copied sorted list and any extra merge structure.

10. LRU Cache (LeetCode 146) — Medium

Method we will use

Hash map for $O(1)$ lookup + ordered structure for $O(1)$ recency updates

- Map: key $\rightarrow$ value (or node).
- Order: most recent at one end, least recent at the other.
- get/put move a key to “most recent”.
- Over capacity $\rightarrow$ delete “least recent”.

In Python, `OrderedDict` is that ordered structure. From scratch it's a doubly linked list.

Using the method with capacity 2

operation	map after	order (LRU $\rightarrow$ MRU)	note
put(1,1)	{1:1}	1	
put(2,2)	{1:1,2:2}	1, 2	
get(1) $\rightarrow$ 1	same	2, 1	method: move 1 to MRU
put(3,3)	{1:1,3:3}	1, 3	method: evict LRU key 2
get(2) $\rightarrow$ -1			2 was evicted

Code

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity: int):
        self.cap = capacity
        self.od = OrderedDict() # first=LRU, last=MRU

    def get(self, key: int) -> int:
        if key not in self.od:
            return -1
        self.od.move_to_end(key) # METHOD: mark MRU
        return self.od[key]

    def put(self, key: int, value: int) -> None:
        if key in self.od:
            self.od.move_to_end(key)
        self.od[key] = value
        if len(self.od) > self.cap:
            self.od.popitem(last=False) # METHOD: evict LRU
```

Alternate — complexity trick**Trick: Ordered dictionary**

- `OrderedDict.move_to_end` performs the recency splice, while `popitem(last=False)` evicts the least-recent key.
- Mention this as production Python; implement the hash map plus DLL when the interview tests data-structure design.

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity):
        self.capacity, self.data = capacity, OrderedDict()
    def get(self, key):
        if key not in self.data: return -1
        self.data.move_to_end(key)
        return self.data[key]
    def put(self, key, value):
        if key in self.data: self.data.move_to_end(key)
        self.data[key] = value
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)
```

Time	$O(1)$ average per operation.
Space	$O(\text{capacity})$.
vs main	Uses a tested built-in; in a handmade DLL, storing each node's key enables $O(1)$ map deletion on eviction.

11. Subarray Sum Equals K (LeetCode 560) — Medium**Method we will use****Running prefix sum + hash map of prefix frequencies**

- If `prefix[j] - prefix[i] = k`, then sum from `i+1..j` equals `k`.
- While scanning, for current prefix `s`, add `count[s-k]` to the answer.
- Then increment `count[s]`.
- Start with `count[0] = 1` (empty prefix).

Using the method on `nums=[1,1,1]`, `k=2`

i	x	s (prefix)	add <code>count[s-k]=count[s-2]</code>	ans	then <code>count[s]++</code>	count map
start		0		0	<code>count[0]=1</code>	<code>{0:1}</code>
0	1	1	<code>count[-1]=0</code>	0	<code>count[1]=1</code>	<code>{0:1,1:1}</code>

i	x	s (prefix)	add count[s-k]=count[s-2]	ans	then count[s]++	count map
1	1	2	count[0]=1	1	count[2]=1	{0:1,1:1,2:1}
2	1	3	count[1]=1	2	count[3]=1	...

Answer **2**. Each time we applied: “how many earlier prefixes equal $s-k$?”

Code

```
from collections import defaultdict

def subarraySum(nums, k):
    count = defaultdict(int)
    count[0] = 1
    s = ans = 0
    for x in nums:
        s += x
        ans += count[s - k]    # METHOD
        count[s] += 1
    return ans
```

Shared imports (problems 12+)

```
from typing import List, Optional
from collections import Counter, defaultdict, deque
import heapq, bisect, random
# Assume ListNode / TreeNode exist as on LeetCode.
```

Alternate — complexity trick

Trick: Prefix existence set

- If the question asks only whether a target-sum subarray exists, store seen prefixes rather than every prefix’s multiplicity.
- Use it for a boolean variant; counting all matching subarrays requires the frequency map.

```
def hasSubarraySum(nums, k):
    prefix, seen = 0, {0}
    for x in nums:
        prefix += x
        if prefix - k in seen:
            return True
        seen.add(prefix)
    return False
```

Time	$O(n)$ expected.
Space	$O(n)$.
vs main	Stores membership only and can return on the first witness.

12. Find the Index of the First Occurrence in a String (LeetCode 28) — Easy

Method we will use

Two pointers with character match rollback (KMP-lite sliding) - Try every start index i in haystack where a match could begin. - Keep a pointer j into **needle**; advance both while characters match. - On mismatch, restart $j = 0$ and move i forward by one (naive but clear). - For interview polish, mention KMP's failure function to avoid redundant rescans. - Return i when $j == \text{len}(\text{needle})$.

Using the method on `haystack = "sadbutsad"`, `needle = "sad"`

```
i=0: h[0]='s' n[0]='s' -> j=1
      h[1]='a' n[1]='a' -> j=2
      h[2]='d' n[2]='d' -> j=3 == len(needle) -> return 0
```

$i=1$ would start later; first hit already found at 0.

Code

```
class Solution:
    def strStr(self, haystack: str, needle: str) -> int:
        if not needle:
            return 0
        n, m = len(haystack), len(needle)
        for i in range(n - m + 1):
            j = 0
            while j < m and haystack[i + j] == needle[j]:
                j += 1
            if j == m:
                return i
        return -1
```

Complexity

Time	$O(n * m)$ naive; $O(n + m)$ with full KMP
Space	$O(1)$ naive; $O(m)$ for KMP failure table

Alternate — complexity trick

Trick: KMP failure links

- The LPS table reuses the longest valid pattern prefix after a mismatch instead of restarting comparisons.
- Use it when worst-case guarantees matter or the text contains highly repetitive prefixes.

```
def strStr(text, pattern):
    if not pattern: return 0
    lps = [0] * len(pattern)
    j = 0
    for i in range(1, len(pattern)):
        while j and pattern[i] != pattern[j]: j = lps[j - 1]
        if pattern[i] == pattern[j]: j += 1
        lps[i] = j
    j = 0
    for i, ch in enumerate(text):
        while j and ch != pattern[j]: j = lps[j - 1]
        if ch == pattern[j]: j += 1
        if j == len(pattern): return i - j + 1
    return -1
```

Time	$O(n + m)$.
------	--------------

Space	$O(m)$.
-------	----------

vs main	Improves naive $O(nm)$ worst-case matching to linear time.
---------	--

What to say

“I slide the needle across the haystack and verify character-by-character. The brute force is acceptable for small inputs; if asked to optimize, I’d build the KMP prefix function so we never move the haystack pointer backward.”

13. Find Peak Element (LeetCode 162) — Medium

Method we will use

Binary search on the mountain shape - A peak satisfies $\text{nums}[\text{mid}] > \text{nums}[\text{mid} \pm 1]$ (with boundary checks). - If $\text{nums}[\text{mid}] < \text{nums}[\text{mid} + 1]$, a peak must exist to the right — search right. - If $\text{nums}[\text{mid}] > \text{nums}[\text{mid} + 1]$, a peak is at mid or left — search left. - $\text{nums}[-1] = \text{nums}[n] = -\infty$ guarantees a peak exists. - Loop until $\text{lo} == \text{hi}$.

Using the method on $\text{nums} = [1, 2, 3, 1]$

$\text{lo}=0$ $\text{hi}=3$ $\text{mid}=1$ $\text{nums}[1]=2 < \text{nums}[2]=3 \rightarrow \text{lo}=2$

$\text{lo}=2$ $\text{hi}=3$ $\text{mid}=2$ $\text{nums}[2]=3 > \text{nums}[3]=1 \rightarrow \text{hi}=2$

$\text{lo}=\text{hi}=2 \rightarrow$ peak index 2 (value 3)

Code

```
class Solution:
    def findPeakElement(self, nums: List[int]) -> int:
```

```

lo, hi = 0, len(nums) - 1
while lo < hi:
    mid = (lo + hi) // 2
    if nums[mid] < nums[mid + 1]:
        lo = mid + 1
    else:
        hi = mid
return lo

```

Complexity

Time	$O(\log n)$
------	-------------

Space	$O(1)$
-------	--------

Alternate — complexity trick

Trick: Slope invariant

- Comparing `nums[mid]` with its right neighbor reveals a side guaranteed to contain a peak, so no sentinels or linear scan are needed.
- Use this to state why discarding half is valid even when the array is not globally sorted.

```

def findPeakElement(nums):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] > nums[mid + 1]:
            hi = mid
        else:
            lo = mid + 1
    return lo

```

Time	$O(\log n)$.
------	---------------

Space	$O(1)$.
-------	----------

vs main	Preserves the optimal bounds; the trick is the local-slope proof, not a second data structure.
---------	--

What to say

“I binary search by comparing `mid` with its right neighbor. The array behaves like a mountain with virtual cliffs at both ends, so one side always leads uphill to a peak.”

14. Largest Number (LeetCode 179) — Medium

Method we will use

Custom sort with concatenation comparator - Sort numbers as strings, but order **a** before **b** if **a+b** > **b+a**. - Example: "9" before "34" because "934" > "349". - After sort, join strings; if result starts with "0", return "0". - Handles negatives? No — all non-negative per problem. - Greedy global order emerges from pairwise comparison.

Using the method on `nums = [3, 30, 34, 5, 9]`

Compare pairs -> order: 9, 5, 34, 3, 30

Join -> "9534330"

Code

```
class Solution:
    def largestNumber(self, nums: List[int]) -> str:
        from functools import cmp_to_key

        def compare(a, b):
            if a + b > b + a:
                return -1
            if a + b < b + a:
                return 1
            return 0

        nums_str = sorted(map(str, nums), key=cmp_to_key(compare))
        ans = "".join(nums_str)
        return "0" if ans[0] == "0" else ans
```

Complexity

Time $O(n \log n * k)$ where k = average digit length

Space $O(n * k)$ for strings

Alternate — complexity trick

Trick: Concatenation comparator

- Order strings **a** and **b** by whether **a+b** or **b+a** is larger; collapse an all-zero result immediately.
- Use it because numeric or lexicographic sorting alone does not encode concatenation order.

```
from functools import cmp_to_key

def largestNumber(nums):
    parts = list(map(str, nums))
    def compare(a, b):
        return (b + a > a + b) - (b + a < a + b)
    parts.sort(key=cmp_to_key(compare))
    return "0" if parts[0] == "0" else "".join(parts)
```

Time	$O(n \log n \cdot L)$ for comparison length L .
Space	$O(nL)$.
vs main	Gets the correct custom order and avoids returning strings like 000.

What to say

“The trick is sorting strings by which concatenation is lexicographically larger. That comparator is transitive enough for Python’s sort and gives the globally largest number.”

15. Identify the Largest Outlier in an Array (LeetCode 3371) — Medium

Method we will use

Total sum + frequency Counter - An outlier x satisfies: $\text{total} - x - x = y$ for some other element y in the array (outlier equals sum of all others). - Equivalently: $x = \text{total} - 2*y$ for some y appearing in `nums`. - Count frequencies so we can remove one copy of y and one of x . - Track the maximum valid outlier. - Iterate each y as the “special sum element.”

Using the method on `nums = [2, 3, 5, 10]`

```
total=20
Try y=2: x=20-4=16 not in array
Try y=3: x=20-6=14 not in array
Try y=5: x=20-10=10 in array, freq ok -> outlier=10
Try y=10: x=0 not present
Answer: 10
```

Code

```
class Solution:
    def getLargestOutlier(self, nums: List[int]) -> int:
        total = sum(nums)
        freq = Counter(nums)
        ans = float("-inf")

        for y in nums:
            x = total - 2 * y
            if x in freq:
                if x != y or freq[y] >= 2:
                    ans = max(ans, x)

        return ans
```

Complexity

Time	$O(n)$
------	--------

Space $O(n)$

Alternate — complexity trick

Trick: Total-sum complement

- For candidate outlier x , the remaining total must be twice a present special-sum value; temporarily remove x to handle duplicates correctly.
- Use it to avoid recomputing sums or rescanning the array for each candidate.

```
from collections import Counter

def getLargestOutlier(nums):
    count, total = Counter(nums), sum(nums)
    answer = float("-inf")
    for x in count:
        count[x] -= 1
        rest = total - x
        if rest % 2 == 0 and count[rest // 2] > 0:
            answer = max(answer, x)
        count[x] += 1
    return answer
```

Time	$O(n)$.
Space	$O(n)$.
vs main	Uses one Counter and one total instead of nested candidate checks.

What to say

“If one number is the outlier, the rest must sum to it. For each candidate y , I compute the implied outlier $\text{total} - 2y$ and verify it exists in the multiset.”

16. Custom Sort String (LeetCode 791) — Medium

Method we will use

Counter + two-pass assembly - Count characters in s . - Walk **order** and append each char $\text{count}[c]$ times, then zero it out. - Append remaining chars from s in any order (often sorted or Counter iteration). - Preserves relative order only as required by **order** priority. - $O(n)$ over alphabet size.

Using the method on **order** = "cba", **s** = "abcd"

```
count: a1 b1 c1 d1
from order: c, b, a -> "cba"
remaining: d -> "cbad"
```

Code

```
class Solution:
    def customSortString(self, order: str, s: str) -> str:
        count = Counter(s)
        res = []
        for ch in order:
            if ch in count:
                res.append(ch * count[ch])
                count[ch] = 0
        for ch, k in count.items():
            res.append(ch * k)
        return "".join(res)
```

Complexity

Time	$O(n +$
Space	$O(1)$ extra (26 letters)

Alternate — complexity trick

Trick: Fixed alphabet counts

- A 26-slot array emits characters in **order** first, then emits every leftover count.
- Use it when inputs are lowercase English letters; use a Counter for a larger or unknown alphabet.

```
def customSortString(order, s):
    count = [0] * 26
    for ch in s: count[ord(ch) - 97] += 1
    out = []
    for ch in order:
        i = ord(ch) - 97
        out.append(ch * count[i])
        count[i] = 0
    for i, freq in enumerate(count):
        out.append(chr(i + 97) * freq)
    return "".join(out)
```

Time	$O(\text{len}(s) + 26)$.
Space	$O(1)$ auxiliary for a fixed alphabet.
vs main	Replaces a hash map or comparison sort with constant-size direct addressing.

What to say

“I count letters in *s*, emit them in the priority of **order**, then dump anything left. It’s a stable priority sort without a full custom comparator.”

17. Design HashMap (LeetCode 706) — Easy

Method we will use

Separate chaining with array of buckets - Fixed array of size m ; each bucket is a list of $[\text{key}, \text{value}]$.
 - $\text{hash}(\text{key}) = \text{key} \% m$; search bucket linearly for key. - **put** updates existing key or appends pair. -
 - **remove** deletes pair from bucket list. - Simple, interview-friendly, no rehashing needed at this scale.

Using the method on **put(1,1)**, **put(2,2)**, **get(1)**, **remove(2)**, **get(2)**

key=1 -> bucket[1%size] -> [(1,1)] get->1
 key=2 -> bucket[2] -> [(2,2)] then removed
 get(2) -> bucket empty -> -1

Code

```
class MyHashMap:
    def __init__(self):
        self.size = 10007
        self.buckets = [[] for _ in range(self.size)]

    def _idx(self, key: int) -> int:
        return key % self.size

    def put(self, key: int, value: int) -> None:
        b = self.buckets[self._idx(key)]
        for i, (k, _) in enumerate(b):
            if k == key:
                b[i] = [key, value]
                return
        b.append([key, value])

    def get(self, key: int) -> int:
        for k, v in self.buckets[self._idx(key)]:
            if k == key:
                return v
        return -1

    def remove(self, key: int) -> None:
        b = self.buckets[self._idx(key)]
        for i, (k, _) in enumerate(b):
            if k == key:
                b.pop(i)
                return
```

Complexity

Time	$O(1)$ average per op; $O(n)$ worst bucket
-------------	--

Space	$O(n + m)$
--------------	------------

Alternate — complexity trick**Trick: Prime bucket count**

- Separate chaining with a prime number of buckets reduces regular collision patterns from structured integer keys.
- Use it to discuss collision behavior; resizing is still needed for production-grade load-factor guarantees.

```
class MyHashMap:
    SIZE = 2069
    def __init__(self):
        self.buckets = [[] for _ in range(self.SIZE)]
    def put(self, key, value):
        bucket = self.buckets[key % self.SIZE]
        for pair in bucket:
            if pair[0] == key: pair[1] = value; return
        bucket.append([key, value])
    def get(self, key):
        for k, value in self.buckets[key % self.SIZE]:
            if k == key: return value
        return -1
    def remove(self, key):
        bucket = self.buckets[key % self.SIZE]
        self.buckets[key % self.SIZE] = [p for p in bucket if p[0] != key]
```

Time	$O(1)$ average, $O(n)$ worst case.
Space	$O(n + \text{buckets})$.
vs main	Lowers avoidable modulo collision patterns while retaining simple chaining.

What to say

“I use modular hashing into buckets with chaining. Collisions are handled by scanning a short list in each bucket — standard separate chaining.”

18. Rotate Image (LeetCode 48) — Medium**Method we will use**

Transpose then reverse each row - 90° clockwise rotation equals transpose + horizontal flip. - Transpose: swap `matrix[i][j]` with `matrix[j][i]` for $j > i$. - Reverse each row in place. - Works in-place without extra matrix. - Alternative: rotate groups of four cells layer by layer.

Using the method on `[[1,2,3],[4,5,6],[7,8,9]]`

transpose:

```
1 4 7
2 5 8
3 6 9
```

reverse rows:

```
7 4 1
8 5 2
9 6 3
```

Code

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        for i in range(n):
            for j in range(i + 1, n):
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
        for row in matrix:
            row.reverse()
```

Complexity

Time $O(n^2)$

Space $O(1)$

Alternate — complexity trick

Trick: Transpose then reverse

- Reflect across the main diagonal, then reverse each row; together those transforms equal a 90° clockwise rotation.
- Use it when the matrix must be changed in place and is square.

```
def rotate(matrix):
    n = len(matrix)
    for r in range(n):
        for c in range(r + 1, n):
            matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
    for row in matrix:
        row.reverse()
```

Time $O(n^2)$.

Space $O(1)$.

vs main Avoids constructing a rotated n-by-n copy.

What to say

“Clockwise 90° is transpose then reverse each row. Two in-place passes, no temp matrix.”

19. Design Tic-Tac-Toe (LeetCode 348) — Medium

Method we will use

Row/col/diagonal running counters - Player 1 adds +1, player 2 adds -1 on each move. - Track rows[r], cols[c], two diagonals. - If any counter reaches $\pm n$, that player wins. - `move` returns winner immediately; O(1) per move. - No need to store the full board.

Using the method on $n=3$, moves: (0,0) P1, (1,1) P2, (0,1) P1, (0,2) P1

(0,0) P1: row0=1
 (1,1) P2: diag1=-1
 (0,1) P1: row0=2
 (0,2) P1: row0=3 -> player 1 wins

Code

```
class TicTacToe:
    def __init__(self, n: int):
        self.n = n
        self.rows = [0] * n
        self.cols = [0] * n
        self.d1 = 0
        self.d2 = 0

    def move(self, row: int, col: int, player: int) -> int:
        val = 1 if player == 1 else -1
        self.rows[row] += val
        self.cols[col] += val
        if row == col:
            self.d1 += val
        if row + col == self.n - 1:
            self.d2 += val

        if (abs(self.rows[row]) == self.n or
            abs(self.cols[col]) == self.n or
            abs(self.d1) == self.n or
            abs(self.d2) == self.n):
            return player
        return 0
```

Complexity

Time	O(1) per move
------	---------------

Space	O(n)
-------	------

Alternate — complexity trick

Trick: Signed line counters

- Add +1 for one player and -1 for the other to row, column, and diagonal totals; absolute value n means a win.
- Use it when moves are valid and the board need not be reconstructed.

```
class TicTacToe:
    def __init__(self, n):
        self.n = n
        self.rows = [0] * n
        self.cols = [0] * n
        self.diag = self.anti = 0
    def move(self, row, col, player):
        delta = 1 if player == 1 else -1
        self.rows[row] += delta; self.cols[col] += delta
        if row == col: self.diag += delta
        if row + col == self.n - 1: self.anti += delta
        return player if self.n in map(abs, (self.rows[row], self.cols[col], self.diag, self.anti))
```

Time	$O(1)$ per move.
Space	$O(n)$.
vs main	Replaces $O(n)$ board scans after every move with four counter updates.

What to say

"I maintain cumulative scores per row, column, and diagonal. A win is detected when any line hits plus or minus n ."

20. Longest Arithmetic Subsequence of Given Difference (LeetCode 1218) — Medium

Method we will use

DP hash: length ending at each value - Let $dp[x]$ = longest chain ending with value x where consecutive diff is difference. - Transition: $dp[x] = dp[x - \text{difference}] + 1$ if predecessor exists, else 1. - Process array left to right updating $dp[nums[i]]$. - Track global maximum. - Works for any difference, positive or negative.

Using the method on $nums = [1,2,3,4]$, $difference = 1$

```
i=0 val=1 dp[1]=1 max=1
i=1 val=2 dp[2]=dp[1]+1=2 max=2
i=2 val=3 dp[3]=3 max=3
i=3 val=4 dp[4]=4 max=4
```

Code

```
class Solution:
    def longestSubsequence(self, nums: List[int], difference: int) -> int:
        dp = {}
```

```

ans = 0
for x in nums:
    prev = dp.get(x - difference, 0)
    dp[x] = prev + 1
    ans = max(ans, dp[x])
return ans

```

Complexity

Time $O(n)$

Space $O(n)$

Alternate — complexity trick

Trick: Rolling value DP

- For each value x , only the best subsequence ending at $x - \text{difference}$ can extend into the best ending at x .
- Use it when order matters but the transition depends on one predecessor value rather than all prior indices.

```

def longestSubsequence(arr, difference):
    best = {}
    answer = 0
    for x in arr:
        best[x] = best.get(x - difference, 0) + 1
        answer = max(answer, best[x])
    return answer

```

Time $O(n)$ expected.

Space $O(u)$, only distinct ending values.

vs main Compresses index DP to one state per value and removes the $O(n^2)$ scan.

What to say

“For each number, I extend the chain ending at $x - \text{diff}$. A hash map stores the best length ending at each value.”

21. Merge Sorted Array (LeetCode 88) — Easy

Method we will use

Two pointers from the end - `nums1` has extra space at the tail; fill from the back to avoid overwriting. - Compare `nums1[i]` and `nums2[j]`; place larger at `write`. - Decrement pointers; copy leftovers from `nums2` if any. - `nums1` leftovers already in place. - In-place $O(1)$ extra space.

Using the method on `nums1 = [1,2,3,0,0,0]`, `m=3`, `nums2 = [2,5,6]`, `n=3`

```
write=5: max(3,6)=6 -> [,, , ,6] j=2
write=4: max(3,5)=5
write=3: max(2,2)=2 tie -> take nums1 i=1
... -> [1,2,2,3,5,6]
```

Code

```
class Solution:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        i, j, write = m - 1, n - 1, m + n - 1
        while j >= 0:
            if i >= 0 and nums1[i] > nums2[j]:
                nums1[write] = nums1[i]
                i -= 1
            else:
                nums1[write] = nums2[j]
                j -= 1
            write -= 1
```

Complexity

Time $O(m + n)$

Space $O(1)$

Alternate — complexity trick

Trick: Fill from the back

- The largest remaining item belongs at the last free slot, so writing backward never overwrites unread values in `nums1`.
- Use it whenever the destination has trailing capacity for an in-place merge.

```
def merge(nums1, m, nums2, n):
    i, j, write = m - 1, n - 1, m + n - 1
    while j >= 0:
        if i >= 0 and nums1[i] > nums2[j]:
            nums1[write] = nums1[i]; i -= 1
        else:
            nums1[write] = nums2[j]; j -= 1
        write -= 1
```

Time $O(m + n)$.

Space $O(1)$.

vs main Avoids an $O(m+n)$ merged copy or costly front insertions.

What to say

“I merge from the rear so unused slots in `nums1` get filled first and nothing gets overwritten before it’s moved.”

22. Maximum Sum Circular Subarray (LeetCode 918) — Medium**Method we will use**

Kadane + total minus minimum subarray - Max subarray in circular array is either: 1. Normal Kadane max on the array, or 2. `total - min_subarray_sum` (wrap-around segment). - Compute Kadane max and Kadane min in one pass each. - Edge case: if all negative, return max element only. - Compare two candidates.

Using the method on `nums = [5,-3,5]`

Kadane max = 7 (linear subarray [5,-3,5])

total=7, min subarray sum = -3

circular candidate = 7 - (-3) = 10 (wrap [5] + [5] skipping -3)

Answer: `max(7, 10) = 10`

Code

```
class Solution:
    def maxSubarraySumCircular(self, nums: List[int]) -> int:
        total = 0
        cur_max = cur_min = 0
        max_sum = min_sum = nums[0]

        for x in nums:
            total += x
            cur_max = max(x, cur_max + x)
            cur_min = min(x, cur_min + x)
            max_sum = max(max_sum, cur_max)
            min_sum = min(min_sum, cur_min)

        if max_sum < 0:
            return max_sum
        return max(max_sum, total - min_sum)
```

Complexity

Time $O(n)$

Space $O(1)$

Alternate — complexity trick

Trick: Dual Kadane pass

- A wrapped maximum equals total sum minus the minimum subarray; compute max, min, and total together.
- Use the non-wrapped maximum when every value is negative, because subtracting the whole minimum would produce an invalid empty subarray.

```
def maxSubarraySumCircular(nums):
    total = 0
    cur_max = best_max = nums[0]
    cur_min = best_min = nums[0]
    for i, x in enumerate(nums):
        if i:
            cur_max = max(x, cur_max + x); best_max = max(best_max, cur_max)
            cur_min = min(x, cur_min + x); best_min = min(best_min, cur_min)
        total += x
    return best_max if best_max < 0 else max(best_max, total - best_min)
```

Time	O(n).
------	-------

Space	O(1).
-------	-------

vs main	Gets both wrapped and ordinary candidates in one pass without doubled arrays.
---------	---

What to say

“The best circular segment is either a normal subarray or everything except the worst middle chunk. Kadane gives max and min subarray sums.”

23. Accounts Merge (LeetCode 721) — Medium

Method we will use

Union-Find on emails - Map each email to an index; union emails belonging to the same account. - Also map email -> person name from input. - After unions, group emails by root parent. - Sort each merged list; output [name] + emails. - Disconnected components = merged accounts.

Using the method on two accounts sharing an email

Account0: John, j@x, j@y
 Account1: John, j@y, j@z
 Union j@x-j@y, j@y-j@z -> one component
 Merge -> John + sorted emails

Code

```
class Solution:
    def accountsMerge(self, accounts: List[List[str]]) -> List[List[str]]:
        parent = {}
        email_to_name = {}
```

```

def find(x):
    parent.setdefault(x, x)
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]

def union(a, b):
    ra, rb = find(a), find(b)
    if ra != rb:
        parent[rb] = ra

for acc in accounts:
    name = acc[0]
    first = acc[1]
    email_to_name[first] = name
    for e in acc[2:]:
        email_to_name[e] = name
        union(first, e)

groups = defaultdict(list)
for e in email_to_name:
    groups[find(e)].append(e)

res = []
for emails in groups.values():
    emails.sort()
    res.append([email_to_name[emails[0]]] + emails)
return res

```

Complexity

Time $O(N \log N)$ for sorting emails; union-find $\sim O(N \cdot N)$

Space $O(N)$

Alternate — complexity trick

Trick: Ranked disjoint sets

- Union emails through account representatives; path compression and union by rank make repeated connectivity operations nearly constant.
- Use it when account-email links form connected components rather than a simple one-pass grouping.

```

def accountsMerge(accounts):
    parent, rank, owner = {}, {}, {}
    def find(x):
        parent.setdefault(x, x)
        if parent[x] != x: parent[x] = find(parent[x])
        return parent[x]
    def union(a, b):

```

```

a, b = find(a), find(b)
if a == b: return
if rank.get(a, 0) < rank.get(b, 0): a, b = b, a
parent[b] = a
rank[a] = max(rank.get(a, 0), rank.get(b, 0) + 1)
for account in accounts:
    for email in account[1:]: owner[email] = account[0]; union(account[1], email)
groups = {}
for email in owner: groups.setdefault(find(email), []).append(email)
return [[owner[root]] + sorted(emails) for root, emails in groups.items()]

```

Time	$O(E(E) + E \log E)$.
------	------------------------

Space	$O(E)$.
-------	----------

vs main	Path compression plus rank prevents tall union-find trees.
---------	--

What to say

“Emails are nodes; accounts connect them. Union-Find merges shared emails, then I bucket by root and sort.”

24. Combination Sum II (LeetCode 40) — Medium

Method we will use

Backtracking + sort + skip duplicates - Sort candidates so equal values are adjacent. - At each index, choose to take or skip; track remaining target. - When taking, loop i..end but skip i if candidates[i]==candidates[i-1] at same depth. - Stop early if candidate > remaining. - Collect when target hits 0.

Using the method on candidates = [10,1,2,7,6,1,5], target = 8

Sorted: [1,1,2,5,6,7,10]

Paths: [1,1,6], [1,2,5], [1,7], [2,6]

Code

```

class Solution:
    def combinationSum2(self, candidates: List[int], target: int) -> List[List[int]]:
        candidates.sort()
        res = []

        def dfs(start, remain, path):
            if remain == 0:
                res.append(path[:])
                return
            for i in range(start, len(candidates)):
                if i > start and candidates[i] == candidates[i - 1]:

```

```

        continue
    if candidates[i] > remain:
        break
    path.append(candidates[i])
    dfs(i + 1, remain - candidates[i], path)
    path.pop()

dfs(0, target, [])
return res

```

Complexity

Time $O(2^n)$ worst case

Space $O(n)$ recursion depth

Alternate — complexity trick

Trick: Sorted duplicate pruning

- Sorting lets one depth skip equal candidates and lets the search stop once a candidate exceeds the remaining target.
- Use the `i > start` duplicate test: equal values may still be selected at different recursion depths.

```

def combinationSum2(candidates, target):
    candidates.sort()
    out = []
    def search(start, remain, path):
        if remain == 0: out.append(path[:]); return
        for i in range(start, len(candidates)):
            if i > start and candidates[i] == candidates[i - 1]: continue
            if candidates[i] > remain: break
            path.append(candidates[i])
            search(i + 1, remain - candidates[i], path)
            path.pop()
        search(0, target, [])
    return out

```

Time	$O(2^n)$ worst case, with substantial pruning.
Space	$O(n)$ recursion excluding output.
vs main	Avoids duplicate result generation and impossible high-value branches.

What to say

“Sort first, use index-based backtracking, and skip duplicate branches at the same tree level to avoid repeated combinations.”

25. Reverse Linked List (LeetCode 206) — Easy

Method we will use

Three pointers iterative - `prev = None`, `curr = head`. - While `curr`: save `nxt`, point `curr.next = prev`, advance `prev` and `curr`. - Return `prev` as new head. - Classic in-place reversal. - Recursive version also fine but iterative is cleaner live.

Using the method on 1 -> 2 -> 3 -> None

```
prev=None curr=1: 1->None, prev=1 curr=2
prev=1 curr=2: 2->1, prev=2 curr=3
prev=2 curr=3: 3->2, prev=3 curr=None
return 3 -> 2 -> 1
```

Code

```
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        prev, curr = None, head
        while curr:
            nxt = curr.next
            curr.next = prev
            prev = curr
            curr = nxt
        return prev
```

Complexity

Time $O(n)$

Space $O(1)$

Alternate — complexity trick

Trick: Iterative pointer reversal

- Carry `prev`, detach the current node toward it, and save `next` before changing the link.
- Use it to avoid recursion depth limits and one stack frame per node.

```
def reverseList(head):
    prev = None
    while head:
        nxt = head.next
        head.next = prev
        prev, head = head, nxt
    return prev
```

Time $O(n)$.

Space $O(1)$.

vs main Removes the recursive solution's $O(n)$ call stack.

What to say

“I walk the list once, rewiring each node to point backward. Three pointers keep track of previous, current, and next.”

26. First Unique Character in a String (LeetCode 387) — Easy

Method we will use

Counter + second pass - Count frequency of each character. - Scan string left to right; return first index with count 1. - If none, return -1. - $O(n)$ time, $O(1)$ space for lowercase 26 letters.

Using the method on `s = "leetcode"`

counts: l1 e3 t1 c1 o1 d1
 scan: index 0 'l' count 1 -> return 0

Code

```
class Solution:
    def firstUniqChar(self, s: str) -> int:
        freq = Counter(s)
        for i, ch in enumerate(s):
            if freq[ch] == 1:
                return i
        return -1
```

Complexity

Time	$O(n)$
-------------	--------

Space	$O(1)$ alphabet
--------------	-----------------

Alternate — complexity trick

Trick: First-index array

- Track each lowercase letter's first index and mark repeated letters with -2; the smallest nonnegative index is the answer.
- Use it for a fixed 26-letter alphabet; a Counter is more general for Unicode.

```
def firstUniqChar(s):
    first = [-1] * 26
    for i, ch in enumerate(s):
        j = ord(ch) - 97
        first[j] = i if first[j] == -1 else -2
```

```
answer = min((i for i in first if i >= 0), default=-1)
return answer
```

Time	$O(n + 26)$.
------	---------------

Space	$O(1)$.
-------	----------

vs main	Uses fixed direct-address storage and avoids a second scan of the full string.
---------	--

What to say

“Two passes: build frequencies, then return the first character that appears exactly once.”

27. Two City Scheduling (LeetCode 1029) — Medium

Method we will use

Sort by cost difference (A - B) ascending - Sending person to A costs **a**; to B costs **b**. - Net extra cost of sending to A vs B is **a - b**. - Sort by **costA - costB ascending** — smallest diff first. - First **n** people go to A; second **n** go to B. - Greedy exchange argument: send people with smallest A-premium to A.

Using the method on costs = [[10,20],[30,200],[400,50],[30,20]]

diffs (A-B): -10, -170, 350, 10

sort asc: -170, -10, 10, 350

A: first 2 -> [30,200] + [10,20] = 30+10 = 40 to A side

B: second 2 -> [400,50] + [30,20] = 50+20 = 70 to B side

total = 40 + 70 = 110

Code

```
class Solution:
    def twoCitySchedCost(self, costs: List[List[int]]) -> int:
        costs.sort(key=lambda x: x[0] - x[1])
        n = len(costs) // 2
        total = 0
        for i in range(n):
            total += costs[i][0] + costs[i + n][1]
        return total
```

Complexity

Time	$O(n \log n)$
------	---------------

Space	$O(1)$
-------	--------

Alternate — complexity trick**Trick: Cost-difference ordering**

- Sending everyone to A gives a baseline; sorting by `costB-costA` chooses the half with the cheapest switches to B.
- Use it to explain the greedy exchange argument: any inverted pair can be swapped without improving cost.

```
def twoCitySchedCost(costs):
    costs.sort(key=lambda c: c[1] - c[0])
    half = len(costs) // 2
    return sum(b for _, b in costs[:half]) + sum(a for a, _ in costs[half:])
```

Time	$O(n \log n)$.
Space	$O(1)$ auxiliary if sorting in place.
vs main	Reduces a two-choice assignment problem to one scalar sort key.

What to say

“People with the smallest A-minus-B difference should fly to A. Sort ascending by `costA - costB`, send the first half to A and the rest to B.”

28. Count Unhappy Friends (LeetCode 1583) — Medium**Method we will use**

Preference rank maps + single forward scan - Build `partner[x]` from pairs and `rank[x][y]` = preference position (lower is better). - For each friend `x`, scan `preferences[x]` until reaching partner `y`. - For every `u` appearing **before** `y` in `x`'s list, if `u` prefers `x` over their own partner, then `x` is unhappy. - Count each unhappy friend once. - No second loop after the partner.

Using the method on $n=4$ paired (0,1),(2,3)

For `x=0`, `partner=1`, scan prefs before 1:

if any `u` ranks `x` above `partner[u]` -> `x` unhappy

Repeat for all `x`

Code

```
class Solution:
    def unhappyFriends(self, n: int, preferences: List[List[int]], pairs: List[List[int]]) -> int:
        partner = {}
        for a, b in pairs:
            partner[a] = b
            partner[b] = a

        rank = [[0] * n for _ in range(n)]
```

```

for i in range(n):
    for pos, j in enumerate(preferences[i]):
        rank[i][j] = pos

unhappy = 0
for x in range(n):
    y = partner[x]
    is_unhappy = False
    for u in preferences[x]:
        if u == y:
            break
        pu = partner[u]
        if rank[u][x] < rank[u][pu]:
            is_unhappy = True
            break
    if is_unhappy:
        unhappy += 1
return unhappy

```

Complexity

Time $O(n^2)$

Space $O(n^2)$

Alternate — complexity trick

Trick: Inverse preference ranks

- Precompute $\text{rank}[x][y]$, turning every ‘does x prefer u to y?’ query from a list scan into $O(1)$.
- Use it whenever the same preference ordering is queried many times.

```

def unhappyFriends(n, preferences, pairs):
    rank = [[0] * n for _ in range(n)]
    partner = [0] * n
    for x in range(n):
        for i, y in enumerate(preferences[x]): rank[x][y] = i
    for x, y in pairs: partner[x], partner[y] = y, x
    unhappy = 0
    for x in range(n):
        y = partner[x]
        unhappy += any(rank[x][u] < rank[x][y] and
                       rank[u][x] < rank[u][partner[u]]
                       for u in preferences[x][:rank[x][y]])
    return unhappy

```

Time $O(n^2)$.

Space $O(n^2)$.

vs mainAvoids repeated $O(n)$ preference searches that can push the check to $O(n^3)$.**What to say**

“Precompute ranks and partners. Friend x is unhappy if someone they prefer over their partner also prefers them back over their own partner.”

29. Find K Closest Elements (LeetCode 658) — Medium**Method we will use**

Binary search for window start - Answer is a contiguous subarray of length k . - Binary search on start index $lo..hi$ where $hi = n - k$. - Compare $arr[mid]$ vs $arr[mid+k]$ distance to x ; shrink window. - Return $arr[lo:lo+k]$. - Alternative: two pointers from ends.

Using the method on $arr = [1,2,3,4,5]$, $k=4$, $x=3$

$lo=0$ $hi=1$ $mid=0$: $x-arr[0]=2$ vs $arr[4]-x=2$ $\rightarrow$ tie, shrink hi
 $lo=1$ $\rightarrow$ window $[2,3,4,5]$

Code

```
class Solution:
    def findClosestElements(self, arr: List[int], k: int, x: int) -> List[int]:
        lo, hi = 0, len(arr) - k
        while lo < hi:
            mid = (lo + hi) // 2
            if x - arr[mid] > arr[mid + k] - x:
                lo = mid + 1
            else:
                hi = mid
        return arr[lo:lo + k]
```

Complexity

Time $O(\log(n - k) + k)$

Space $O(1)$

Alternate — complexity trick**Trick: Binary-search the window**

- The answer is a contiguous length- k window; compare the two boundary losses to decide whether its start lies left or right.
- Use it because selecting k items individually wastes the input's sorted structure.

```
def findClosestElements(arr, k, x):
    lo, hi = 0, len(arr) - k
    while lo < hi:
        mid = (lo + hi) // 2
        if x - arr[mid] > arr[mid + k] - x:
            lo = mid + 1
        else:
            hi = mid
    return arr[lo:lo + k]
```

Time	$O(\log(n-k) + k)$ including output.
-------------	--------------------------------------

Space	$O(k)$ output, $O(1)$ auxiliary.
--------------	----------------------------------

vs main	Improves expansion or sorting approaches by locating the whole window directly.
----------------	---

What to say

“The k closest elements form a length- k window. I binary search the best left boundary by comparing edge distances to x .”

30. Range Sum of BST (LeetCode 938) — Easy

Method we will use

Pruning DFS - If `node.val` in `[low, high]`, add to sum. - If `node.val > high`, go left (smaller values might qualify). - If `node.val < low`, go right. - BST ordering lets us skip whole subtrees. - Recursive or iterative stack.

Using the method on BST root `[10,5,15,3,7,null,18]`, `low=7`, `high=15`

10 in range +10, go left and right

5: skip left, right 7 ok +7

15 in +15

Sum: $10+7+15=32$

Code

```
class Solution:
    def rangeSumBST(self, root: Optional[TreeNode], low: int, high: int) -> int:
        if not root:
            return 0
        ans = 0
        if low <= root.val <= high:
            ans += root.val
        if root.val > low:
            ans += self.rangeSumBST(root.left, low, high)
        if root.val < high:
            ans += self.rangeSumBST(root.right, low, high)
        return ans
```

```

        ans += self.rangeSumBST(root.right, low, high)
    return ans

```

Complexity

Time $O(n)$ worst; often less with pruning

Space $O(h)$ recursion

Alternate — complexity trick

Trick: BST range pruning

- If a node is below **low**, discard its entire left subtree; if above **high**, discard its entire right subtree.
- Use it only because BST ordering proves those skipped branches cannot contribute.

```

def rangeSumBST(root, low, high):
    if not root: return 0
    if root.val < low:
        return rangeSumBST(root.right, low, high)
    if root.val > high:
        return rangeSumBST(root.left, low, high)
    return (root.val + rangeSumBST(root.left, low, high)
            + rangeSumBST(root.right, low, high))

```

Time	$O(n)$ worst case, often proportional to visited/pruned nodes.
Space	$O(h)$ recursion.
vs main	Can skip whole subtrees instead of visiting every node.

What to say

“I DFS but prune: if the node is too small I only need the right subtree; if too large, only left.”

31. Search in Rotated Sorted Array (LeetCode 33) — Medium

Method we will use

Modified binary search - One half of $[lo..hi]$ is always sorted. - If target lies in sorted half, search there; else the other half. - Compare $nums[mid]$ with $nums[lo]$ to find sorted side. - Standard **while** $lo \leq hi$. - No duplicates in this version.

Using the method on $nums = [4,5,6,7,0,1,2]$, $target = 0$

```

lo=0 hi=6 mid=3 val=7, left [4..7] sorted, 0 not in -> lo=4
lo=4 hi=6 mid=5 val=1, left [0,1] sorted, 0 in -> hi=4
lo=4 hi=4 found index 4

```

Code

```

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        lo, hi = 0, len(nums) - 1
        while lo <= hi:
            mid = (lo + hi) // 2
            if nums[mid] == target:
                return mid
            if nums[lo] <= nums[mid]:
                if nums[lo] <= target < nums[mid]:
                    hi = mid - 1
                else:
                    lo = mid + 1
            else:
                if nums[mid] < target <= nums[hi]:
                    lo = mid + 1
                else:
                    hi = mid - 1
        return -1

```

Complexity

Time	$O(\log n)$
------	-------------

Space	$O(1)$
-------	--------

Alternate — complexity trick

Trick: Sorted-half elimination

- At least one half around mid is sorted; test whether the target belongs in that half before discarding the other.
- Use the inclusive boundary checks carefully; this version assumes distinct values.

```

def search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target: return mid
        if nums[lo] <= nums[mid]:
            if nums[lo] <= target < nums[mid]: hi = mid - 1
            else: lo = mid + 1
        else:
            if nums[mid] < target <= nums[hi]: lo = mid + 1
            else: hi = mid - 1
    return -1

```

Time	$O(\log n)$.
------	---------------

Space $O(1)$.

vs main Retains binary-search complexity despite the rotation.

What to say

“At each step I identify which half is sorted and check whether the target can live there.”

32. Word Search (LeetCode 79) — Medium

Method we will use

DFS backtracking on grid - For each cell matching first letter, run DFS with visited marking. - Directions: 4-neighbors; temporarily mark cell, undo on backtrack. - Base case: matched all chars in word. - Prune if char mismatch. - Can mutate board with # or use a set.

Using the method on board with word "ABCCED"

Start at 'A', explore neighbors matching 'B', mark visited, recurse to 'C', etc.
Backtrack unmark if path fails.

Code

```
class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        rows, cols = len(board), len(board[0])

        def dfs(r, c, i):
            if i == len(word):
                return True
            if r < 0 or c < 0 or r >= rows or c >= cols:
                return False
            if board[r][c] != word[i]:
                return False
            tmp, board[r][c] = board[r][c], '#'
            found = (dfs(r + 1, c, i + 1) or dfs(r - 1, c, i + 1) or
                    dfs(r, c + 1, i + 1) or dfs(r, c - 1, i + 1))
            board[r][c] = tmp
            return found

        for r in range(rows):
            for c in range(cols):
                if dfs(r, c, 0):
                    return True
        return False
```

Complexity

Time $O(m * n * 4^L)$
Space $O(L)$ recursion

Alternate — complexity trick

Trick: In-place visitation mark

- Temporarily replace a used board cell with #, recurse, then restore it during backtracking.
- Use it when board mutation is allowed during the call; restoration preserves the caller-visible input.

```
def exist(board, word):
    rows, cols = len(board), len(board[0])
    def dfs(r, c, i):
        if i == len(word): return True
        if not (0 <= r < rows and 0 <= c < cols) or board[r][c] != word[i]:
            return False
        saved, board[r][c] = board[r][c], "#"
        found = any(dfs(r + dr, c + dc, i + 1)
                    for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)))
        board[r][c] = saved
        return found
    return any(dfs(r, c, 0) for r in range(rows) for c in range(cols))
```

Time $O(\text{rows} \cdot \text{cols} \cdot 4)$.

Space $O(L)$ recursion; $O(1)$ visited storage.

vs main Removes the per-path visited set.

What to say

“I try every starting cell and DFS with backtracking, marking cells visited and restoring them when retreating.”

33. Remove All Occurrences of a Substring (LeetCode 1910) — Medium

Method we will use

Stack as string builder - Scan `s` left to right, push chars onto stack. - After each push, if stack top matches `part`, pop `len(part)` chars. - Stack contents at end form result. - Handles overlapping removals (e.g. "aaaa" removing "aa"). - Equivalent to repeated find/replace from left.

Using the method on `s = "daabcbaabcbc"`, `part = "abc"`

```
build until ... daabc -> see abc at end pop -> dab
continue -> dababc -> pop abc -> dab
continue -> dabc -> pop -> d
```

Code

```
class Solution:
    def removeOccurrences(self, s: str, part: str) -> str:
        stack = []
        m = len(part)
        for ch in s:
            stack.append(ch)
            if len(stack) >= m and "".join(stack[-m:]) == part:
                for _ in range(m):
                    stack.pop()
        return "".join(stack)
```

Complexity

Time $O(n * m)$ with join check; $O(n)$ with rolling hash/KMP

Space $O(n)$

Alternate — complexity trick

Trick: Suffix-only stack check

- After appending one character, only the stack's newest m characters can form a new occurrence of `part`.
- Use it for online removal; repeated global `replace` calls rescan old text.

```
def removeOccurrences(s, part):
    stack, m = [], len(part)
    for ch in s:
        stack.append(ch)
        if len(stack) >= m and "".join(stack[-m:]) == part:
            del stack[-m:]
    return "".join(stack)
```

Time	$O(nm)$ with direct suffix comparison.
-------------	--

Space	$O(n)$.
--------------	----------

vs main	Avoids repeatedly rescanning the entire remaining string; a KMP-state stack can make it $O(n)$.
----------------	--

What to say

“I simulate building the string with a stack and delete whenever the tail matches `part` — that naturally handles overlaps.”

34. Longest Increasing Subsequence (LeetCode 300) — Medium

Method we will use

DP $O(n^2)$ with patience sorting note - $dp[i]$ = LIS ending at i ; $dp[i] = 1 + \max(dp[j])$ for $j < i$ and $nums[j] < nums[i]$. - Track global max. - Optimized: patience sorting — $tails[k]$ = smallest tail of LIS length $k+1$; binary search each num $\rightarrow O(n \log n)$. - Mention both in interview.

Using the method on `nums = [10,9,2,5,3,7,101,18]`

`dp` ends: 1,1,1,2,2,3,4,4

max length = 4 e.g. [2,3,7,101]

Code

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        if not nums:
            return 0
        n = len(nums)
        dp = [1] * n
        for i in range(n):
            for j in range(i):
                if nums[j] < nums[i]:
                    dp[i] = max(dp[i], dp[j] + 1)
        return max(dp)
```

Complexity

Time $O(n^2)$ shown; $O(n \log n)$ with patience + bisect

Space $O(n)$

Alternate — complexity trick

Trick: Patience tails

- `tails[i]` stores the smallest possible tail of an increasing subsequence of length $i+1$; binary-search where each value belongs.
- Use `bisect_left` for strictly increasing LIS and `bisect_right` for nondecreasing LIS.

```
from bisect import bisect_left

def lengthOfLIS(nums):
    tails = []
    for x in nums:
        i = bisect_left(tails, x)
        if i == len(tails): tails.append(x)
        else: tails[i] = x
    return len(tails)
```

Time	$O(n \log n)$.
Space	$O(n)$.
vs main	Improves the index-pair DP from $O(n^2)$ to $O(n \log n)$.

What to say

“I start with the classic DP where each index extends all smaller predecessors. If we need faster, patience sorting with binary search on tail arrays gives $O(n \log n)$.”

35. Move Pieces to Obtain a String (LeetCode 2337) — Medium

Method we will use

Two pointers skipping _ on both strings - Loop: advance *i* and *j* past '_' on *start* and *target*. - If one string is exhausted before the other, return False. - Compare non-blank chars; they must match. - 'L' can only move left: require *start_idx* $\geq$ *target_idx* (*i* $\geq$ *j*). - 'R' can only move right: require *start_idx* $\leq$ *target_idx* (*i* $\leq$ *j*). - Both exhausted together $\rightarrow$ True.

Using the method on *start* = "_L__R__R_", *target* = "L_____RR"

```
i=1 L, j=0 L: i>=j ok (1>=0)
skip blanks, match R at i=4 j=7: i<=j ok (4<=7)
match R at i=7 j=8: i<=j ok
both done -> True
```

Code

```
class Solution:
    def canChange(self, start: str, target: str) -> bool:
        i = j = 0
        n, m = len(start), len(target)
        while i < n or j < m:
            while i < n and start[i] == '_':
                i += 1
            while j < m and target[j] == '_':
                j += 1
            if i == n or j == m:
                return i == n and j == m
            if start[i] != target[j]:
                return False
            if start[i] == 'L' and i < j:
                return False
            if start[i] == 'R' and i > j:
                return False
            i += 1
            j += 1
        return True
```

Complexity

Time $O(n)$

Space $O(1)$

Alternate — complexity trick

Trick: Ignore-space alignment

- Two pointers visit only L and R; corresponding pieces must match, and L may only move left while R may only move right.
- Use it to avoid simulating every legal move or building intermediate strings.

```
def canChange(start, target):
    a = [(ch, i) for i, ch in enumerate(start) if ch != "_"]
    b = [(ch, i) for i, ch in enumerate(target) if ch != "_"]
    if [ch for ch, _ in a] != [ch for ch, _ in b]: return False
    for (ch, i), (_, j) in zip(a, b):
        if ch == "L" and i < j: return False
        if ch == "R" and i > j: return False
    return True
```

Time $O(n)$.

Space $O(n)$ here; $O(1)$ with streaming pointers.

vs main Replaces move simulation with invariant checks; a streaming implementation uses constant space.

What to say

“I skip blanks on both strings in sync. L pieces must not be right of their target; R pieces must not be left of their target.”

36. Search a 2D Matrix (LeetCode 74) — Medium

Method we will use

Binary search on virtual 1D array - Matrix sorted row-major with each row first $>$ previous row last. - Index mid maps to `matrix[mid // n][mid % n]`. - Standard binary search for target. - $O(\log(mn))$ time. - No extra flattening needed.

Using the method on `matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, `target = 3`

```
lo=0 hi=11 mid=5 -> matrix[1][1]=11 > 3 -> hi=4
mid=2 -> 5 > 3 -> hi=1
mid=0 -> 1 < 3 -> lo=1
mid=1 -> 3 found
```

Code

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        if not matrix:
            return False
        m, n = len(matrix), len(matrix[0])
        lo, hi = 0, m * n - 1
        while lo <= hi:
            mid = (lo + hi) // 2
            val = matrix[mid // n][mid % n]
            if val == target:
                return True
            if val < target:
                lo = mid + 1
            else:
                hi = mid - 1
        return False
```

Complexity

Time	$O(\log(mn))$
------	---------------

Space	$O(1)$
-------	--------

Alternate — complexity trick

Trick: Flattened binary search

- Map virtual index i to `matrix[i // cols][i % cols]`, treating the ordered matrix as one sorted array.
- Use it when each row starts after the previous row ends; row-and-column sorted matrices need a different search.

```
def searchMatrix(matrix, target):
    rows, cols = len(matrix), len(matrix[0])
    lo, hi = 0, rows * cols - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        value = matrix[mid // cols][mid % cols]
        if value == target: return True
        if value < target: lo = mid + 1
        else: hi = mid - 1
    return False
```

Time	$O(\log(\text{rows} \cdot \text{cols}))$.
------	--

Space	$O(1)$.
-------	----------

vs main	Uses one binary search rather than searching rows separately.
---------	---

What to say

“I treat the matrix as a sorted array of length $m \times n$ and binary search with index-to-cell conversion.”

37. Search Suggestions System (LeetCode 1268) — Medium**Method we will use**

Sort products + binary search prefix range - Sort products. - For each prefix of `searchWord`, binary search lower bound of prefix. - Take up to 3 lexicographically smallest from that position if they share prefix. - Alternatively trie; sort+bisect is simpler to code. - Stop early when 3 found.

Using the method on `products = ["mobile","mouse","moneypot","monitor"]`, `searchWord = "mouse"`

```
"m": suggestions mobile, monitor, moneypot
"mo": mobile, moneypot, monitor
"mou": mouse
"mous": mouse
"mouse": mouse
```

Code

```
class Solution:
    def suggestedProducts(self, products: List[str], searchWord: str) -> List[List[str]]:
        products.sort()
        res = []
        lo = 0
        prefix = ""
        for ch in searchWord:
            prefix += ch
            lo = bisect.bisect_left(products, prefix, lo)
            hi = bisect.bisect_left(products, prefix + '{', lo)
            res.append(products[lo:min(lo + 3, hi)])
        return res
```

Complexity

Time	$O(P \log P + L * \log P)$ where $L = \text{searchWord length}$
Space	$O(1)$ extra

Alternate — complexity trick**Trick: Bisected prefix range**

- After sorting products, binary-search the first product for each growing prefix and inspect only the next three entries.
- Use it for a static catalog; a trie is preferable for many dynamic prefix queries.

```

from bisect import bisect_left

def suggestedProducts(products, searchWord):
    products.sort()
    answer, start, prefix = [], 0, ""
    for ch in searchWord:
        prefix += ch
        start = bisect_left(products, prefix, start)
        answer.append([p for p in products[start:start + 3]
                       if p.startswith(prefix)])
    return answer

```

Time $O(n \log n + m \log n)$, excluding small output checks.

Space $O(1)$ auxiliary beyond output if sorting in place.

vs main Avoids scanning every product for every prefix.

What to say

“Sort once, then for each typed character binary search the prefix lower bound and grab up to three matches.”

38. Binary Search Tree Iterator (LeetCode 173) — Medium

Method we will use

Controlled stack inorder - Push all left spine from root onto stack in constructor/`_push_left`. - `next()`: pop top (current smallest), then push left chain of its right child. - `hasNext()`: stack non-empty. - Amortized $O(1)$ per next. - Lazy traversal — $O(h)$ space.

Using the method on BST [7,3,15,null,null,9,20]

```

init: stack push 7,3
next -> 3; push nothing from right null
next -> 7; push 15,9
next -> 9
next -> 15; push 20
next -> 20

```

Code

```

class BSTIterator:
    def __init__(self, root: Optional[TreeNode]):
        self.stack = []
        self._left(root)

    def _left(self, node):
        while node:
            self.stack.append(node)

```

```

        node = node.left

    def next(self) -> int:
        node = self.stack.pop()
        if node.right:
            self._left(node.right)
        return node.val

    def hasNext(self) -> bool:
        return bool(self.stack)

```

Complexity

Time $O(1)$ amortized **next**; $O(h)$ init

Space $O(h)$

Alternate — complexity trick

Trick: Lazy left spine

- Keep only the path to the next inorder node; after popping it, push the left spine of its right subtree.
- Use it to explain why each node is pushed and popped once, giving amortized $O(1)$ **next**.

```

class BSTIterator:
    def __init__(self, root):
        self.stack = []
        self._push_left(root)
    def _push_left(self, node):
        while node:
            self.stack.append(node)
            node = node.left
    def next(self):
        node = self.stack.pop()
        self._push_left(node.right)
        return node.val
    def hasNext(self):
        return bool(self.stack)

```

Time $O(1)$ amortized **next**, $O(1)$ **hasNext**.

Space $O(h)$.

vs main Avoids materializing all n inorder values.

What to say

“I simulate inorder with a stack of left paths. Each **next** pops the smallest and advances to the next inorder node.”

39. Minimum Operations to Make Character Frequencies Equal (LeetCode 3389) — Hard

Interview sketch note: The full LC Hard problem typically requires a more nuanced DP over frequency multisets (choosing which letters survive, target frequency per survivor, and min insert/delete/relabel cost). The code below is a **simplified enumeration sketch** — state the approach and complexity honestly; refine if the interviewer gives constraints.

Method we will use

Try each target frequency + cost from Counter (heuristic sweep) - Only lowercase letters; bounded alphabet (26). - Try each candidate uniform frequency f that survivors might share. - Cost = sum of $|\text{count}[c] - f|$ for kept letters, plus adjustments for total length. - Allow deleting letters entirely by treating excess as removal cost. - Pick minimum over candidate f values. - Full solution may need DP over sorted frequency counts.

Using the method on $s = \text{"aaabc"}$

```
counts a3 b1 c1 n=5
try f=1: reduce a by 2 -> cost 2
try f=2: add/remove to balance -> compute min
pick min over candidate f
```

Code

```
class Solution:
    def minOperations(self, s: str) -> int:
        cnt = Counter(s)
        counts = list(cnt.values())
        n = len(s)
        ans = float("inf")

        for f in range(0, max(counts) + 1):
            cost = 0
            kept = 0
            for c in cnt.values():
                if c > f:
                    cost += c - f
                    kept += f
                else:
                    cost += f - c
                    kept += f
            if kept != n:
                cost += abs(kept - n)
            ans = min(ans, cost)

        return ans
```

Complexity

Time	$O(26 * \text{max_freq})$ — sketch only; full LC Hard may be higher
Space	$O(1)$

Alternate — complexity trick

Trick: Enumerate target frequency

- The alphabet is only 26 letters, so try every target frequency and use DP across adjacent letters to account for delete, insert, or increment-to-next-letter operations.
- Use it to turn an unbounded-looking target choice into at most $\text{max}(\text{count})$ small 26-state evaluations.

```
from collections import Counter

def makeStringGood(s):
    freq = [Counter(s)[chr(97 + i)] for i in range(26)]
    answer = len(s)
    for target in range(1, max(freq) + 1):
        dp = [0] * 27
        for i in range(25, -1, -1):
            dp[i] = min(freq[i], abs(freq[i] - target)) + dp[i + 1]
            if i < 25 and freq[i + 1] < target:
                next_deficit = target - freq[i + 1]
                movable = freq[i] if freq[i] <= target else freq[i] - target
                dp[i] = min(dp[i], max(next_deficit, movable) + dp[i + 2])
        answer = min(answer, dp[0])
    return answer
```

Time	$O(26 \cdot \text{max_frequency})$.
Space	$O(26)$.
vs main	Enumerates bounded targets and models adjacent relabeling instead of using an insert/delete-only heuristic.

What to say

“I’d explain that the real problem needs DP over frequency distributions. As a sketch, I enumerate target frequencies and sum insert/delete costs — honest about this being a heuristic unless we build the full state machine.”

40. Random Pick with Weight (LeetCode 528) — Medium

Method we will use

Prefix sums + binary search - Build prefix array where each slot spans weight proportion. - Draw r in $[1, \text{total}]$ uniformly. - Binary search first index where $\text{prefix}[i] \geq r$. - Return that index. - pickIndex $O(\log n)$ after $O(n)$ preprocess.

Using the method on `w = [1,3]`

`prefix = [1,4]`, `total=4`

`r=1` -> index 0; `r=3` -> index 1 (covers 1..3)

Code

```
class Solution:
    def __init__(self, w: List[int]):
        self.prefix = []
        run = 0
        for x in w:
            run += x
            self.prefix.append(run)
        self.total = run

    def pickIndex(self) -> int:
        target = random.randint(1, self.total)
        lo, hi = 0, len(self.prefix) - 1
        while lo < hi:
            mid = (lo + hi) // 2
            if self.prefix[mid] < target:
                lo = mid + 1
            else:
                hi = mid
        return lo
```

Complexity

Time `O(n)` init; `O(log n)` per pick

Space `O(n)`

Alternate — complexity trick

Trick: Prefix roulette

- Map each weight to an interval in a prefix sum, draw one integer uniformly, then use `bisect_left` to locate its interval.
- Use it when picks are frequent and weights stay fixed between picks.

```
from bisect import bisect_left
from random import randint
```

```
class Solution:
    def __init__(self, w):
        self.prefix = []
        total = 0
        for weight in w:
            total += weight
            self.prefix.append(total)
```

```
def pickIndex(self):
    return bisect_left(self.prefix, randint(1, self.prefix[-1]))
```

Time	$O(n)$ setup, $O(\log n)$ per pick.
Space	$O(n)$.
vs main	Makes each weighted sample logarithmic instead of scanning all weights.

What to say

“I prefix-sum the weights into ranges and binary search a random draw — classic weighted roulette selection.”

41. Design Hit Counter (LeetCode 362) — Medium

Method we will use

Circular buffer of 300 buckets - Timestamp space modulo 300 seconds. - `bucket[i]` stores count for second `i`; `times[i]` stores which second. - On hit: if bucket stale, reset count; increment. - `getHits`: sum buckets whose time within last 300 seconds. - $O(1)$ hit; $O(300)$ query.

Using the method on hits at $t=1,2,300,300$

`bucket` stores counts per second mod 300

`getHits(300)` sums buckets with time in $(0, 300]$

Code

```
class HitCounter:
    def __init__(self):
        self.times = [0] * 300
        self.hits = [0] * 300

    def hit(self, timestamp: int) -> None:
        idx = timestamp % 300
        if self.times[idx] != timestamp:
            self.times[idx] = timestamp
            self.hits[idx] = 1
        else:
            self.hits[idx] += 1

    def getHits(self, timestamp: int) -> int:
        total = 0
        for i in range(300):
            if timestamp - self.times[i] < 300:
                total += self.hits[i]
        return total
```

Complexity

Time $O(1)$ hit; $O(300)$ getHits

Space $O(300)$

Alternate — complexity trick

Trick: Fixed 300-second ring

- Reuse slots by `timestamp % 300`, resetting a slot whenever its stored timestamp is stale.
- Use it because the time window is fixed; for arbitrary windows, use a deque of timestamp counts.

```
class HitCounter:
    def __init__(self):
        self.time = [0] * 300
        self.count = [0] * 300
    def hit(self, timestamp):
        i = timestamp % 300
        if self.time[i] != timestamp:
            self.time[i], self.count[i] = timestamp, 0
        self.count[i] += 1
    def getHits(self, timestamp):
        return sum(c for t, c in zip(self.time, self.count)
                    if timestamp - t < 300)
```

Time $O(1)$ hit and $O(300)=O(1)$ query.

Space $O(300)=O(1)$.

vs main Bounds memory independently of the number of hits.

What to say

“Only the last 300 seconds matter, so I keep a ring buffer indexed by `timestamp % 300` and sum valid buckets on query.”

42. Remove Stones to Minimize the Total (LeetCode 1962) — Medium

Method we will use

Max heap (simulate largest piles first) - Always halve the largest pile (ceil division) for max reduction.
 - Use max-heap via negating values in Python `heapq`. - Repeat k times; sum heap. - Greedy: reducing big piles first minimizes total.

Using the method on `piles = [5,4,9]`, $k = 2$

```
heap max 9 -> 5 piles [5,4,5]
heap max 5 -> 3 piles [3,4,5]
sum=12
```

Code

```
class Solution:
    def minStoneSum(self, piles: List[int], k: int) -> int:
        heap = [-p for p in piles]
        heapq.heapify(heap)
        for _ in range(k):
            x = -heapq.heappop(heap)
            heapq.heappush(heap, -((x + 1) // 2))
        return -sum(heap)
```

Complexity

Time	$O(k \log n)$
-------------	---------------

Space	$O(n)$
--------------	--------

Alternate — complexity trick

Trick: Negated max heap

- Each operation has the greatest payoff on the current largest pile, so repeatedly pop and reinsert that pile through a max heap.
- Use it when k is much smaller than sorting after every operation.

```
import heapq

def minStoneSum(piles, k):
    heap = [-pile for pile in piles]
    heapq.heapify(heap)
    for _ in range(k):
        pile = -heapq.heappop(heap)
        heapq.heappush(heap, -(pile - pile // 2))
    return -sum(heap)
```

Time	$O(n + k \log n)$.
-------------	---------------------

Space	$O(n)$.
--------------	----------

vs main	Avoids $O(k n \log n)$ repeated full sorting.
----------------	---

What to say

“Each operation should hit the current largest pile. A max-heap makes that easy — pop, halve with ceiling, push back.”

43. Remove Nth Node From End of List (LeetCode 19) — Medium

Method we will use

Two pointers with gap n - Advance **fast** n steps ahead. - Move **fast** and **slow** until **fast** reaches end; **slow** before target. - Delete **slow.next**. - Dummy node handles removing head. - One pass.

Using the method on 1->2->3->4->5, n=2

fast moves 2: stops at node3
 move both until fast.next is None
 slow at 3, delete 4 -> 1->2->3->5

Code

```
class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
        dummy = ListNode(0, head)
        fast = slow = dummy
        for _ in range(n):
            fast = fast.next
        while fast.next:
            fast = fast.next
            slow = slow.next
        slow.next = slow.next.next
        return dummy.next
```

Complexity

Time O(L)

Space O(1)

Alternate — complexity trick

Trick: Fixed pointer gap

- Advance **fast** by n+1, then move both pointers until **fast** ends; **slow.next** is exactly the node to remove.
- Use a dummy node so deleting the head needs no special case.

```
def removeNthFromEnd(head, n):
    dummy = ListNode(0, head)
    slow = fast = dummy
    for _ in range(n + 1):
        fast = fast.next
    while fast:
        slow, fast = slow.next, fast.next
    slow.next = slow.next.next
    return dummy.next
```

Time	$O(\text{length})$.
Space	$O(1)$.
vs main	Uses one pass instead of first measuring the list length.

What to say

“I offset fast by n nodes so when fast hits the end, slow sits just before the node to remove.”

44. Simplify Path (LeetCode 71) — Medium

Method we will use

Stack of directory names - Split on '/'; ignore empty and '.'. - On '..', pop if stack non-empty. - Else push directory name. - Join with '/' prefixed by '/'. - Handles absolute Unix paths.

Using the method on path = "/home//foo/./../bar"

```
parts: home foo .. bar
stack: home -> home foo -> home -> home bar
result: /home/bar
```

Code

```
class Solution:
    def simplifyPath(self, path: str) -> str:
        stack = []
        for part in path.split('/'):
            if part == '' or part == '.':
                continue
            if part == '..':
                if stack:
                    stack.pop()
            else:
                stack.append(part)
        return '/' + '/'.join(stack)
```

Complexity

Time $O(n)$

Space $O(n)$

Alternate — complexity trick

Trick: Canonical component stack

- Ignore empty and . components, pop for .., and push ordinary directory names.

- Use split components rather than character parsing; repeated slashes then disappear naturally.

```
def simplifyPath(path):
    stack = []
    for part in path.split("/"):
        if not part or part == ".":
            continue
        if part == "..":
            if stack: stack.pop()
        else:
            stack.append(part)
    return "/" + "/".join(stack)
```

Time	$O(n)$.
------	----------

Space	$O(n)$.
-------	----------

vs main	Normalizes the path in one pass without repeated string rewrites.
---------	---

What to say

“Split on slashes and simulate cd with a stack — pop on parent directory, skip current and empty segments.”

45. Zero Array Transformation I (LeetCode 3355) — Medium

Method we will use

Difference array (range add) - Each $[l, r]$ adds 1 to range — use diff: $\text{diff}[l] += 1$, $\text{diff}[r+1] -= 1$.
 - Prefix sum diff to get actual increments at each index. - Check whether each index received enough decrements to zero $\text{nums}[i]$. - Standard range update trick.

Using the method on $\text{nums} = [1, 2, 3]$, queries covering ranges

Apply diff marks, prefix $\rightarrow$ increments per index

Verify $\text{ops}[i] \geq \text{nums}[i]$ for all i

Code

```
class Solution:
    def isZeroArray(self, nums: List[int], queries: List[List[int]]) -> bool:
        n = len(nums)
        diff = [0] * (n + 1)
        for l, r in queries:
            diff[l] += 1
            diff[r + 1] -= 1
        ops = 0
        for i in range(n):
            ops += diff[i]
            if ops < nums[i]:
```

```

        return False
    return True

```

Complexity

Time $O(n + q)$

Space $O(n)$

Alternate — complexity trick

Trick: Difference coverage

- Mark each range with +1 at its start and -1 after its end; one prefix sum gives total available decrements per index.
- Use it when every query contributes uniformly across a range and query order does not matter.

```

def isZeroArray(nums, queries):
    diff = [0] * (len(nums) + 1)
    for left, right in queries:
        diff[left] += 1
        diff[right + 1] -= 1
    available = 0
    for i, need in enumerate(nums):
        available += diff[i]
        if available < need:
            return False
    return True

```

Time $O(n + q)$.

Space $O(n)$.

vs main Improves direct $O(nq)$ range application to one query pass plus one prefix pass.

What to say

“Range increments are applied with a difference array; one prefix pass tells how many times each index was covered.”

46. Sqrt(x) (LeetCode 69) — Easy

Method we will use

Binary search on answer - Search integer `ans` in $[0, x]$ such that $ans * ans \leq x$ and $(ans+1)^2 > x$.
 - Mid squares compare to x ; shrink left or right. - Avoid floating point. - Return `hi` when loop ends.

Using the method on $x = 8$

```

lo=0 hi=8 mid=4 16>8 hi=3
mid=1 ok lo=2
mid=2 4<=8 lo=3
mid=3 9>8 hi=2 -> answer 2

```

Code

```

class Solution:
    def mySqrt(self, x: int) -> int:
        if x < 2:
            return x
        lo, hi = 1, x // 2
        while lo <= hi:
            mid = (lo + hi) // 2
            if mid * mid <= x:
                lo = mid + 1
            else:
                hi = mid - 1
        return hi

```

Complexity

Time	$O(\log x)$
Space	$O(1)$

Alternate — complexity trick**Trick: Integer Newton iteration**

- Newton's update $(r + x/r)/2$ rapidly converges from above while staying in integer arithmetic.
- Use it as the numerical alternative to binary search; stop when the next estimate no longer decreases.

```

def mySqrt(x):
    if x < 2: return x
    r = x
    while r > x // r:
        r = (r + x // r) // 2
    return r

```

Time	$O(\log \log x)$ iterations for fixed-width integers.
Space	$O(1)$.
vs main	Converges faster in practice than $O(\log x)$ binary search while avoiding floats.

What to say

“I binary search the largest integer whose square is at most x — standard integer sqrt template.”

47. All Nodes Distance K in Binary Tree (LeetCode 863) — Medium

Method we will use

Parent map + BFS from target - DFS to record `parent[node]` for every node. - BFS from `target` distance 0, tracking visited (including parents). - When distance == k, collect values. - Treat tree as undirected graph.

Using the method on target node 5, k=2

Build parents, BFS:

`dist0 {5}, dist1 {2,6,parents}, dist2 {1,3,7,...}`

collect vals at dist k

Code

```
class Solution:
    def distanceK(self, root: TreeNode, target: TreeNode, k: int) -> List[int]:
        parent = {}

        def dfs(node, par):
            if not node:
                return
            parent[node] = par
            dfs(node.left, node)
            dfs(node.right, node)

        dfs(root, None)
        q = deque([(target, 0)])
        seen = {target}
        ans = []

        while q:
            node, d = q.popleft()
            if d == k:
                ans.append(node.val)
                continue
            for nxt in (node.left, node.right, parent.get(node)):
                if nxt and nxt not in seen:
                    seen.add(nxt)
                    q.append((nxt, d + 1))

        return ans
```

Complexity

Time	$O(n)$
Space	$O(n)$

Alternate — complexity trick**Trick: Undirected tree BFS**

- Build parent pointers once, then treat left, right, and parent as neighbors in a BFS from the target.
- Use it because downward-only tree traversal cannot reach cousins through ancestors.

```
from collections import deque

def distanceK(root, target, k):
    parent = {}
    def link(node, par=None):
        if not node: return
        parent[node] = par
        link(node.left, node); link(node.right, node)
    link(root)
    q, seen = deque([(target, 0)]), {target}
    answer = []
    while q:
        node, dist = q.popleft()
        if dist == k: answer.append(node.val); continue
        for nxt in (node.left, node.right, parent[node]):
            if nxt and nxt not in seen:
                seen.add(nxt); q.append((nxt, dist + 1))
    return answer
```

Time	$O(n)$.
------	----------

Space	$O(n)$.
-------	----------

vs main	Turns the tree into an implicit graph and stops expanding at distance k.
---------	--

What to say

“I add parent pointers, then BFS outward from target like an undirected graph until depth k.”

48. The kth Factor of n (LeetCode 1492) — Medium**Method we will use**

Iterate i from 1 to $\sqrt{n}$ - If i divides n, record factor i; if $i \neq n/i$, also record n/i . - Sort collected factors; return kth (1-indexed) or -1. - Early exit possible if list long enough. - $O(\sqrt{n})$ enumeration.

Using the method on $n = 12$, $k = 3$

i=1 -> 1,12

i=2 -> 2,6

i=3 -> 3,4

sorted [1,2,3,4,6,12] -> 3rd is 3

Code

```
class Solution:
    def kthFactor(self, n: int, k: int) -> int:
        factors = []
        for i in range(1, int(n ** 0.5) + 1):
            if n % i == 0:
                factors.append(i)
                if i * i != n:
                    factors.append(n // i)
        factors.sort()
        return factors[k - 1] if k <= len(factors) else -1
```

Complexity

Time $O(\sqrt{n} \log \sqrt{n})$ for sort

Space $O(\sqrt{n})$

Alternate — complexity trick

Trick: Paired square-root factors

- Each divisor below $\sqrt{n}$ contributes a paired divisor above it; collect small factors and reverse the large half.
- Use it when k may be far smaller than n but factor order must remain increasing.

```
def kthFactor(n, k):
    small, large = [], []
    d = 1
    while d * d <= n:
        if n % d == 0:
            small.append(d)
            if d * d != n: large.append(n // d)
        d += 1
    factors = small + large[::-1]
    return factors[k - 1] if k <= len(factors) else -1
```

Time	$O(\sqrt{n})$.
-------------	-----------------

Space	$O(\text{number of factors})$.
--------------	---------------------------------

vs main	Improves a scan through n to a square-root factor enumeration.
----------------	--

What to say

“I enumerate divisors up to $\sqrt{n}$, pairing i with n/i , sort, and pick the k th.”

49. Continuous Subarray Sum (LeetCode 523) — Medium

Method we will use

Prefix mod + hash map - Subarray sum multiple of k same prefix remainder mod k at two indices. - Map remainder $\rightarrow$ earliest index. - If same remainder seen at least 2 apart, return True. - Initialize $\{0: -1\}$ for subarrays from start. - Requires length ≥ 2 .

Using the method on `nums = [23,2,4,6,7]`, `k = 6`

prefix mods: $23\%6=5$, $25\%6=1$, $29\%6=5$ at index 0 and 2 $\rightarrow$ distance ≥ 2 true

Code

```
class Solution:
    def checkSubarraySum(self, nums: List[int], k: int) -> bool:
        rem_to_idx = {0: -1}
        run = 0
        for i, x in enumerate(nums):
            run += x
            if k != 0:
                run %= k
            if run in rem_to_idx:
                if i - rem_to_idx[run] >= 2:
                    return True
            else:
                rem_to_idx[run] = i
        return False
```

Complexity

Time	$O(n)$
Space	$O(\min(n, k))$

Alternate — complexity trick

Trick: Earliest prefix remainder

- Equal prefix sums modulo k enclose a subarray divisible by k ; retain the earliest index to maximize length.
- Initialize remainder 0 at index -1 and require an index gap of at least two.

```
def checkSubarraySum(nums, k):
    first = {0: -1}
    prefix = 0
    for i, x in enumerate(nums):
        prefix += x
        rem = prefix % k if k else prefix
        if rem in first:
            if i - first[rem] >= 2: return True
```

```

    else:
        first[rem] = i
    return False

```

Time	$O(n)$ expected.
Space	$O(\min(n,$
vs main	Replaces checking all $O(n^2)$ subarrays with one modular prefix pass.

What to say

“Equal prefix remainders mod k mean the subarray between them sums to a multiple of k . I store the first index of each remainder.”

50. Best Time to Buy and Sell Stock IV (LeetCode 188) — Hard

Method we will use

DP with transaction states - At most k transactions; split days and transactions. - $\text{buy}[t]$ = max profit after at most t transactions, currently holding stock. - $\text{sell}[t]$ = max profit after at most t transactions, not holding. - For each price: update sells then buys from previous day. - If $k \geq n/2$, reduce to unlimited (Stock II).

Using the method on small example $k=2$

Day by day:

```

sell[t] = max(sell[t], buy[t]+price)
buy[t]  = max(buy[t], sell[t-1]-price)
Track max sell[k]

```

Code

```

class Solution:
    def maxProfit(self, k: int, prices: List[int]) -> int:
        if not prices or k == 0:
            return 0
        n = len(prices)
        if k >= n // 2:
            profit = 0
            for i in range(1, n):
                if prices[i] > prices[i - 1]:
                    profit += prices[i] - prices[i - 1]
            return profit

        sell = [0] * (k + 1)
        buy = [float("-inf")] * (k + 1)
        for p in prices:
            for t in range(1, k + 1):

```

```

        sell[t] = max(sell[t], buy[t] + p)
        buy[t] = max(buy[t], sell[t - 1] - p)
    return sell[k]

```

Complexity

Time $O(n * k)$

Space $O(k)$

Alternate — complexity trick

Trick: Unlimited-transactions shortcut

- When $k \geq n/2$, transaction limits cannot bind, so sum every positive day-to-day increase.
- Check this branch before allocating $O(k)$ DP; otherwise use rolling buy/sell states.

```

def maxProfit(k, prices):
    if k >= len(prices) // 2:
        return sum(max(0, b - a) for a, b in zip(prices, prices[1:]))
    buy = [float("-inf")] * (k + 1)
    sell = [0] * (k + 1)
    for price in prices:
        for t in range(1, k + 1):
            buy[t] = max(buy[t], sell[t - 1] - price)
            sell[t] = max(sell[t], buy[t] + price)
    return sell[k]

```

Time $O(n)$ in the unlimited case; otherwise $O(nk)$.

Space $O(k)$, or $O(1)$ for the shortcut.

vs main Avoids wasteful $O(nk)$ DP when k is effectively unlimited.

What to say

“I DP over number of completed transactions. Two arrays track best profit holding vs not holding; if k is large enough it becomes unlimited-transaction greedy.”

51. Palindromic Substrings (LeetCode 647) — Medium

Method we will use

Expand around centers - Each center is char i or gap $i, i+1$. - Expand while equal; count palindromes found. - $O(n^2)$ time, $O(1)$ space. - Alternative: DP table $dp[i][j]$.

Using the method on $s = "abc"$

centers expand:

$a, b, c, ab, bc \rightarrow 3$ single chars = 3 palindromic substrings

Code

```
class Solution:
    def countSubstrings(self, s: str) -> int:
        n = len(s)
        ans = 0

        def expand(l, r):
            nonlocal ans
            while l >= 0 and r < n and s[l] == s[r]:
                ans += 1
                l -= 1
                r += 1

        for i in range(n):
            expand(i, i)
            expand(i, i + 1)
        return ans
```

Complexity

Time $O(n^2)$

Space $O(1)$

Alternate — complexity trick

Trick: Expand around centers

- Every palindrome has one of $2n-1$ odd/even centers; expand while the mirrored characters match.
- Use it when only the count is needed and no $O(n^2)$ palindrome table must be retained.

```
def countSubstrings(s):
    answer = 0
    for center in range(2 * len(s) - 1):
        left = center // 2
        right = left + center % 2
        while left >= 0 and right < len(s) and s[left] == s[right]:
            answer += 1
            left -= 1; right += 1
    return answer
```

Time $O(n^2)$.

Space $O(1)$.

vs main Keeps DP's time bound but removes its $O(n^2)$ table.

What to say

“Every palindrome has a center. I expand outward for odd and even lengths and count valid expansions.”

52. Generate Parentheses (LeetCode 22) — Medium

Method we will use

Backtracking with open/close counts - Add '(' if open < n. - Add ')' if close < open. - When length 2n, append to result. - Ensures validity by construction. - DFS or iterative with queue.

Using the method on n = 3

build to length 6:

((())) ((())() ()()) ()()() ((()))

Code

```
class Solution:
    def generateParenthesis(self, n: int) -> List[str]:
        res = []

        def dfs(path, open_cnt, close_cnt):
            if len(path) == 2 * n:
                res.append("".join(path))
                return
            if open_cnt < n:
                path.append('(')
                dfs(path, open_cnt + 1, close_cnt)
                path.pop()
            if close_cnt < open_cnt:
                path.append(')')
                dfs(path, open_cnt, close_cnt + 1)
                path.pop()

        dfs([], 0, 0)
        return res
```

Complexity

Time $O(4^n / \sqrt{n})$ Catalan structures

Space $O(n)$ recursion

Alternate — complexity trick

Trick: Valid-prefix generation

- Add (only while fewer than n are open and add) only while closes are fewer than opens.
- Use it to generate only valid prefixes instead of producing all $2^{(2n)}$ strings and filtering.

```
def generateParenthesis(n):
    answer = []
```

```
def build(path, opened, closed):
    if len(path) == 2 * n:
        answer.append(path); return
    if opened < n:
        build(path + "(", opened + 1, closed)
    if closed < opened:
        build(path + ")", opened, closed + 1)
build("", 0, 0)
return answer
```

Time $O(C_n \cdot n)$, proportional to Catalan-sized output.

Space $O(n)$ recursion excluding output.

vs main Prunes every prefix that can never become balanced.

What to say

“I backtrack adding opens while under n and closes while $closes < opens$ — only valid strings get built.”

53. Container With Most Water (LeetCode 11) — Medium

Method we will use

Two pointers from ends - $lo=0, hi=n-1$; $area = \min(h[lo], h[hi]) * (hi-lo)$. - Move the shorter line inward (only way to maybe increase area). - Track max area. - Greedy proof: shorter side is the bottleneck.

Using the method on `height = [1,8,6,2,5,4,8,3,7]`

$lo=0$ $hi=8$ $area=8*1=8$ move lo ($1<7$)

...

max area 49 at indices 1 and 8

Code

```
class Solution:
    def maxArea(self, height: List[int]) -> int:
        lo, hi = 0, len(height) - 1
        ans = 0
        while lo < hi:
            ans = max(ans, min(height[lo], height[hi]) * (hi - lo))
            if height[lo] < height[hi]:
                lo += 1
            else:
                hi -= 1
        return ans
```

Complexity

Time $O(n)$

Space $O(1)$

Alternate — complexity trick

Trick: Move the shorter wall

- Area is limited by the shorter wall, so moving the taller wall inward cannot improve that pair; discard the shorter side.
- Use the dominance proof to justify two pointers rather than checking all pairs.

```
def maxArea(height):
    left, right, answer = 0, len(height) - 1, 0
    while left < right:
        answer = max(answer, (right - left) * min(height[left], height[right]))
        if height[left] <= height[right]:
            left += 1
        else:
            right -= 1
    return answer
```

Time $O(n)$.

Space $O(1)$.

vs main Improves brute-force $O(n^2)$ pair enumeration to one pass.

What to say

“I two-pointer from both ends and always discard the shorter wall — width shrinks but height can only rise if we move the short side.”

54. Spiral Matrix (LeetCode 54) — Medium

Method we will use

Boundary shrink simulation - Four bounds: top, bottom, left, right. - Traverse top row $L \rightarrow R$, right col $T \rightarrow B$, bottom $R \rightarrow L$, left col $B \rightarrow T$; shrink bounds. - Stop when $top > bottom$ or $left > right$. - Collect in order.

Using the method on `matrix = [[1,2,3],[4,5,6],[7,8,9]]`

top row 1,2,3 | right 6,9 | bottom 8,7 | left 4 | center 5

output [1,2,3,6,9,8,7,4,5]

Code

```

class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        if not matrix:
            return []
        top, bottom = 0, len(matrix) - 1
        left, right = 0, len(matrix[0]) - 1
        res = []
        while top <= bottom and left <= right:
            for c in range(left, right + 1):
                res.append(matrix[top][c])
            top += 1
            for r in range(top, bottom + 1):
                res.append(matrix[r][right])
            right -= 1
            if top <= bottom:
                for c in range(right, left - 1, -1):
                    res.append(matrix[bottom][c])
                bottom -= 1
            if left <= right:
                for r in range(bottom, top - 1, -1):
                    res.append(matrix[r][left])
                left += 1
        return res

```

Complexity

Time	$O(mn)$
Space	$O(1)$ extra

Alternate — complexity trick

Trick: Shrinking boundaries

- Maintain top, bottom, left, and right bounds; after traversing one edge, shrink it and guard the remaining reverse edges.
- Use it to avoid a visited matrix; boundary checks handle single remaining rows or columns.

```

def spiralOrder(matrix):
    out = []
    top, bottom, left, right = 0, len(matrix)-1, 0, len(matrix[0])-1
    while top <= bottom and left <= right:
        out += matrix[top][left:right+1]; top += 1
        for r in range(top, bottom+1): out.append(matrix[r][right])
        right -= 1
        if top <= bottom:
            out += matrix[bottom][left:right+1][::-1]; bottom -= 1
        if left <= right:
            for r in range(bottom, top-1, -1): out.append(matrix[r][left])

```

```

        left += 1
    return out

```

Time $O(\text{rows} \cdot \text{cols})$.

Space $O(1)$ auxiliary beyond output.

vs main Eliminates visited-state storage.

What to say

“I peel the matrix layer by layer, shrinking top/bottom/left/right after each side.”

55. Gas Station (LeetCode 134) — Medium

Method we will use

Greedy one pass - If total gas < total cost, impossible → -1. - Track current tank; if tank < 0, reset start to next station. - Final start is valid if total sum ≥ 0. - Single $O(n)$ scan.

Using the method on gas = [1,2,3,4,5], cost = [3,4,5,1,2]

total surplus may be negative → -1

(or positive case: tank dips → reset start)

Code

```

class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        if sum(gas) < sum(cost):
            return -1
        start = 0
        tank = 0
        for i in range(len(gas)):
            tank += gas[i] - cost[i]
            if tank < 0:
                start = i + 1
                tank = 0
        return start

```

Complexity

Time $O(n)$

Space $O(1)$

Alternate — complexity trick

Trick: Deficit reset

- If the running tank becomes negative at i , no station since the current start can reach $i+1$, so reset the candidate to $i+1$.
- First verify total gas covers total cost; that condition guarantees the final candidate succeeds.

```
def canCompleteCircuit(gas, cost):
    total = tank = start = 0
    for i, (gain, spend) in enumerate(zip(gas, cost)):
        delta = gain - spend
        total += delta; tank += delta
        if tank < 0:
            start, tank = i + 1, 0
    return start if total >= 0 else -1
```

Time	$O(n)$.
------	----------

Space	$O(1)$.
-------	----------

vs main	Replaces trying every start in $O(n^2)$ with one greedy elimination pass.
---------	---

What to say

“If total gas covers total cost, a unique start exists. I track running surplus and restart candidate start whenever tank goes negative.”

56. Zigzag Iterator (LeetCode 281) — Medium

Method we will use

Queue of [vector, index] pairs - Store each non-empty vector with index 0 in a deque. - `next()`: pop front `[vec, idx]`, return `vec[idx]`, increment `idx`; if more elements, append back to queue. - `hasNext()`: queue non-empty. - Round-robin across vectors.

Using the method on `v1 = [1,2]`, `v2 = [3,4,5,6]`

order: 1,3,2,4,5,6 (zigzag across vectors)

Code

```
class ZigzagIterator:
    def __init__(self, v1: List[int], v2: List[int]):
        self.q = deque()
        if v1:
            self.q.append([v1, 0])
        if v2:
            self.q.append([v2, 0])

    def next(self) -> int:
        vec, idx = self.q.popleft()
        val = vec[idx]
```

```

        idx += 1
        if idx < len(vec):
            self.q.append([vec, idx])
        return val

    def hasNext(self) -> bool:
        return len(self.q) > 0

```

Complexity

Time $O(1)$ amortized per next

Space $O(k)$ vectors

Alternate — complexity trick

Trick: Queue active indices

- Queue (vector, index) states; after yielding one item, requeue only if that vector still has another item.
- Use it to generalize naturally from two vectors to any number of iterables.

```

from collections import deque

class ZigzagIterator:
    def __init__(self, v1, v2):
        self.q = deque((v, 0) for v in (v1, v2) if v)
    def next(self):
        vector, i = self.q.popleft()
        value = vector[i]
        if i + 1 < len(vector):
            self.q.append((vector, i + 1))
        return value
    def hasNext(self):
        return bool(self.q)

```

Time	$O(1)$ per item.
-------------	------------------

Space	$O(\text{number of active vectors})$.
--------------	--

vs main	Avoids empty-vector special cases and supports k-way zigzag iteration.
----------------	--

What to say

“I keep a queue of (vector, index) pairs and rotate after each output — simple round-robin zigzag.”

57. Group the People Given the Group Size They Belong To (LeetCode 1282) — Medium

Method we will use

Map `groupSize` → list of ids - For each person `i` with required size `g`, append `i` to `groups[g]`. - When `groups[g]` length hits `g`, push to answer and reset bucket. - Exact fit — no sorting needed. - $O(n)$ time.

Using the method on `groupSizes = [3,3,3,3,3,1,3]`

size3 bucket fills [0,1,2] → output, then [3,4,6], etc.
size1 [5]

Code

```
class Solution:
    def groupThePeople(self, groupSizes: List[int]) -> List[List[int]]:
        buckets = defaultdict(list)
        res = []
        for i, g in enumerate(groupSizes):
            buckets[g].append(i)
            if len(buckets[g]) == g:
                res.append(buckets[g])
                buckets[g] = []
        return res
```

Complexity

Time $O(n)$

Space $O(n)$

Alternate — complexity trick

Trick: Flush full buckets

- Accumulate indices by required size and emit a group immediately when that size's bucket fills.
- Use it because every bucket is guaranteed to partition exactly under the problem constraints.

```
def groupThePeople(groupSizes):
    buckets, answer = {}, []
    for person, size in enumerate(groupSizes):
        bucket = buckets.setdefault(size, [])
        bucket.append(person)
        if len(bucket) == size:
            answer.append(bucket[:])
            bucket.clear()
    return answer
```

Time

$O(n)$.

Space	$O(n)$ including output; at most unfinished bucket storage beyond it.
vs main	Builds valid groups online without sorting people by size.

What to say

“I bucket people by required group size and flush a group whenever the bucket is full.”

58. Top K Frequent Words (LeetCode 692) — Medium

Method we will use

Counter + custom sort or min-heap - Count word frequencies. - Sort by $(-freq, word)$ for lex tie-break. - Take first k . - Heap variant: maintain size- k heap for streaming.

Using the method on `words = ["i", "love", "leetcode", "i", "love", "coding"]`, `k=2`

counts: i2 love2 leetcode1 coding1
sorted by freq then alpha -> i, love

Code

```
class Solution:
    def topKFrequent(self, words: List[str], k: int) -> List[str]:
        cnt = Counter(words)
        return sorted(cnt.keys(), key=lambda w: (-cnt[w], w))[:k]
```

Complexity

Time $O(N \log N)$ for sort; $O(N \log k)$ with heap

Space $O(N)$

Alternate — complexity trick

Trick: Size- k reverse heap

- Keep only k candidates; encode lexical ties in a reversed string wrapper so the least desirable top- k word is evicted.
- Use a heap when k is much smaller than the number of distinct words; full sorting is simpler when k is large.

```
from collections import Counter
import heapq

class Rev(str):
    def __lt__(self, other):
        return str.__gt__(self, other)
```

```
def topKFrequent(words, k):
    heap = []
    for word, freq in Counter(words).items():
        heapq.heappush(heap, (freq, Rev(word), word))
        if len(heap) > k: heapq.heappop(heap)
    return [item[2] for item in sorted(heap, key=lambda x: (-x[0], x[2]))]
```

Time	$O(n + u \log k + k \log k)$.
Space	$O(u)$ counts plus $O(k)$ heap.
vs main	Reduces ranking from $O(u \log u)$ full sort to $O(u \log k)$.

What to say

“Count frequencies, then sort with key negative frequency and alphabetical word for tie-breaks, and slice top k .”

Study note

Problems 12–58 cover the core C3-style toolkit: binary search variants, two pointers, heaps, union-find, backtracking, prefix/difference arrays, and design questions with $O(1)$ or amortized tricks. For each problem, lead with the invariant (“what stays true each step”), walk one tiny example on the board, then code. Pair this sheet with timed reps: explain method $\rightarrow$ example $\rightarrow$ code $\rightarrow$ complexity in under 15 minutes per medium. Hard ones (188, 3389): state the DP/state definition clearly even if you simplify implementation — interviewers reward correct structure over perfect optimization.

How to study (so this sticks)

For every problem above:

1. Read the **method** name.
2. Cover the code.
3. Redo the “**Using the method**” table on paper.
4. Only then look at the code and check it matches your table.
5. Change the example numbers and repeat the table.

That is what the interview is: **name a method, then apply it to the input out loud.**

Full C3 AI coding hit-list. Problem 1 is the deepest template — every other section follows method $\rightarrow$ apply on example $\rightarrow$ code.